



INSTITUTO TECNOLÓGICO JOSÉ MARIO MOLINA PASQUEL Y HENRÍQUEZ

OSIRIS BAYRIKI ROSAS OCHOA
ESTRUCTURAS DE DATOS

MANUAL DE PRACTICAS

Contenido

PRACTICA 1	5
Agregar un numero en cadena y mandarlo a llamar	5
PRACTICA 2	6
Arreglos, Listas, Ciclos y condiciones	6
PRACTICA 3	7
Leer 10 números y almacenarlos en un Arreglo	7
PRACTICA 4 TAREA	8
Leer 10 números y almacenarlos en una Lista, Validar números y sumar los números agregados por el usuario	8
PRACTICA 5	10
10 elementos, Números en arreglo, Caracteres en Listas y muestra cuantos elementos hay en cada estructura	10
PRACTICA 6	12
Pide lo mismo que la P5 pero con método principal	12
PRACTICA 7	14
Crear una lista donde se almacenará 5 elementos pidiendo Nombre, Edad y Sexo	14
PRACTICA 8	16
Leer una cadena y que muestre en pantalla cuantos números tiene, cuantas mayúsculas, minúsculas y espacios	16
PRACTICA 9 TAREA	18
Crear una lista, no contener espacio, mayúscula 1ra letra, contener todas las vocales. 5 elementos	18
PRACTICA 10	21
Pide lo mismo que la P9 pero otra solución agregando Validaciones	21
PRACTICA 11 REPASO	24
Almacenar Nombre, Precio de producto y calcular el costo (12% y 16%) y precio de venta	24
PRACTICA 12 REPASO	26
Formula General	26
PRACTICA 13 REPASO	28
Leer dato cualquiera y almacenarlo en Lista (Entero y Decimal) con try y ValueError	28
PARCIAL 2	31

PRACTICA 1	32
Pide 5 calificaciones y agrega a una lista	32
PRACTICA 2	34
Pide 5 calificaciones y al final se muestra la lista de las calificaciones y la suma total del promedio general, valida los números en otro archivo	34
PRACTICA 3	38
Pide usuario y contraseña mediante cajas de texto, label y botón con mensajes de error y correcto	38
PRACTICA 4	41
Pide usuario y contraseña mediante cajas de texto, label y botón con mensajes de error y correcto con clases	41
PRACTICA 5	45
3 Calificaciones con lista, orden descendente	45
PRACTICA 6	51
4 calificaciones los suma y agrega lista, también promedia en resultado general de los números ingresados	51
PRACTICA 7	56
Ingresa dos números, se agregan en lista y obtiene el número de mayor y menor	56
PRACTICA 8	63
Ingresa dos números random y obtiene el número mayor y menor	63
PRACTICA 9	68
Validar números, letras y signos	68
REPASO 1	71
REPASO 2	75
REPASO 3	80
REPASO 4	81
La palabra debe contener todas las vocales	81
PARCIAL 3	83
PRACTICA 1	84
RadioButton con ordenar método Pilas y colas, Eliminar método Burbuja o selección	84
PRACTICA 2	88
Realizar RFC Validando letras y números, agregándolos en la lista y ordenar en método pila o cola	88
PRACTICA 3	93

Agregar Nombre, Teléfono y Domicilio a una lista y agregar su sexo con un RadioButton	93
PRACTICA 4	97
Agregar Nombre, Edad y Correo a la tabla generando una Clave, al seleccionar podrás editar el registro	97
PARCIAL 4	101
PRACTICA 1	102
Escribir 2 números, botón enviar para ir a otra ventana y sumar los dos números ingresados	102
PRACTICA 2	105
Ingresar Usuario y password, botón para ir a otra ventana y a través de una base de datos guarda los usuarios ingresados puedes: modificar, eliminar y agregar	105
PRACTICA 3	114
Ingresar un número que desee y envía a otra ventana el número y sus repeticiones	114
PRACTICA 4	117
Tabla donde agregas producto, descripción, precio y cantidad agregando un ID y código el cual también puedes	117
modificar o eliminar	117
EXTRAS	124
Operaciones en python	124
REPASO	126
GUIA DE CODIGOS	129

PRACTICA 1

Agregar un numero en cadena y mandarlo a llamar

```
'''Mostramos en pantalla un mensaje con print, agregamos una variable para que  
almacene el dato que ingresara el usuario con el input y el mensaje para que ingrese  
el dato que pide el programa, al final imprime el dato que ingreso el usuario llamando  
a la variable 'a' lo que se mostrara en la consola'''
```

```
print ('Escribe un numero')  
#Muestra en pantalla el mensaje  
#Displays the message on the screen  
a = input('Escribe un numero')  
#La variable 'a' almacena lo que ingrese el usuario con el input() y su mensaje  
#The variable 'a' stores what the user enters with input() and its message  
print(a)  
#Imprime en pantalla lo que el usuario escribió y que fue guardado en la variable 'a'  
#Prints on the screen what the user typed and was stored in the variable 'a'
```

```
Escribe un numero  
Escribe un numero2  
2
```

PRACTICA 2

Arreglos, Listas, Ciclos y condiciones

```
1  '''Almacenar datos en un Arreglo
2  Almacenar datos en una lista
3  Ciclos y condiciones '''
4
5  '''Store data in an Array
6  Store data in a List
7  Loops and conditions '''
8
9  '''Tipos de Arreglo'''
10 '''Array Types'''
11 a = [10]
12 #Aquí creas una lista llamada 'a' con un único elemento: '[10]'.
13 #Here you create a list called 'a' with a single element: '[10]'.
14 a[0] = 10
15 #Arreglo con posición '0 a 10'
16 #Array with position '0 to 10'
17 a[1] = 10
18 #Arreglo con posición 1 a 10
19 #Array with position a to 10
20
21 '''Tipos de Lista'''
22 '''List Types'''
23 b = []
24 #LISTA RECOMENDABLE Creas la lista
25 #RECOMMENDED LIST You create the list
26 b = {'hola', 10, 10.05, False, {1, 2, 3, 4}}
27 #GUARDAR Los sets en Python no permiten elementos repetidos y no tienen orden fijo.
28 #STORE Sets in Python do not allow duplicate elements and have no fixed order.
29
30 #CICLOS Y CONDICIONES
31 #LOOPS AND CONDITIONS
32 if (len(a) > len(b)):
33     #Mide cuántos elementos hay en la lista entre a y b,
34     #dependiendo de cuál sea más grande, imprime el mensaje correspondiente.
35     #Measures how many elements are in the list between a and b,
36     #depending on which one is bigger, it prints the corresponding message.
37     print('A es mayor')
38     #Mensaje si a es mayor a b
39     #Message if a is greater than b
40 else:
41     #Si no sale que b es mayor
42     #If not, it means b is greater
43     print('B es mayor')
44     #Mensaje si b es mayor a
45     #Message if b is greater than a
46
47 #CICLO FOR
48 #FOR LOOP
49 for i in a:
50     #Recorre cada elemento de la lista 'a' y lo guarda en la variable 'i'
51     #Goes through each element of the list 'a' and stores it in the variable 'i'
52     print(i)
53     #Imprime la lista i
54     #Prints the element i
```

PRACTICA 3

Leer 10 números y almacenarlos en un Arreglo

```
''' Hacer un programa que lea 10 numeros y los alamacene
en un arreglo'''
''' Make a program that reads 10 numbers and stores them
in an array'''

a = [0,0,0,0,0,0,0,0,0,0]
#Arreglo con variable llamada 'a' que almacena 10 elementos
#Array with variable called 'a' that stores 10 elements

for i in range(0,10):
#Ciclo for que tendra un rango de 0 al 9
#For loop that will have a range from 0 to 9
    a[i] = int(input(f'Escribe un numero {i}: '))
    #Ingresa un numero el usuario con 'input()'
    #User enters a number with 'input()'
    #int() Convierte el texto que ingresa a numero entero
    #int() converts the entered text into an integer
    #a[i] Guardar el dato en cada posición del arreglo
    #a[i] stores the data in each position of the array
for i in a:
#Recorre todos los elementos de la lista a
#Iterates through all the elements of the list 'a'
    print(i)
    #Imprime cada número en pantalla.
    #Prints each number on the screen
```

```
Escribe un numero 0: 1
Escribe un numero 1: 2
Escribe un numero 2: 3
Escribe un numero 3: 4
Escribe un numero 4: 5
Escribe un numero 5: 6
Escribe un numero 6: 7
Escribe un numero 7: 8
Escribe un numero 8: 9
Escribe un numero 9: 10
1
2
3
4
5
6
7
8
9
10
```

PRACTICA 4 TAREA

Leer 10 números y almacenarlos en una Lista, Validar números y sumar los números agregados por el usuario

```
'''Hacer un programa que lea 10 numeros y los alamacene
en una lista'''
'''Make a program that reads 10 numbers and stores them
in a list'''

a = []
#Crea una lista vacía llamada a donde se guardarán los números válidos que el usuario introduce.
#Creates an empty list called a where the valid numbers entered by the user will be stored.
s = 0
#Inicia la variable s en 0. Se usará como acumulador para sumar los números guardados.
#Initializes the variable s to 0. It will be used as an accumulator to sum the stored numbers.
n = 0
#Contador n en 0. Cuenta cuántos números válidos se han guardado en a
#Counter n set to 0. Counts how many valid numbers have been stored in a
numeros = "0123456789";
#Cadena con los caracteres permitidos para considerar una entrada como número.
#String with the allowed characters to consider an entry as a number.

while(n < 10):
#Bucle while que se repite mientras n sea menor que 10
#While loop that repeats while n is less than 10
    b = input('Escribe un numero: ')
    #Pide al usuario que escriba algo y lo guarda como cadena en b
    #Asks the user to enter something and stores it as a string in b
    x = 0
    #x servirá para contar cuántos caracteres de la cadena b son dígitos
    #x will be used to count how many characters of the string b are digits
    for i in b:
    #RECORRE LOS ELEMENTOS B
    #ITERATES THROUGH THE ELEMENTS OF b
        if i in numeros:
            x += 1
            #Si el carácter i está en la cadena numeros se incrementa x
            #If the character i is in the string numeros, x is incremented
    if len(b) == x:
    #Si son iguales, se considera la entrada válida como número entero.
    #If they are equal, the entry is considered valid as an integer number.
        a.append(int(b))
        #Convierte la cadena b a entero con int(b) y la añade a la lista a.
        #Converts the string b to an integer with int(b) and adds it to the list a.
        n += 1
        #Incrementa n porque se guardó un número válido.
        #Increments n because a valid number was stored.
    else:
        print('El valor no es numero ')
    #Si la entrada no está compuesta únicamente por dígitos, muestra el mensaje de error
    #If the entry is not composed only of digits, displays the error message
```

```
for i in a:
    print(i)
    s += i
#Despues de ingresar los 10 numeros recorre lista a imprime el número
#(print(i)) y lo suma al acumulador s (s += i).
#After entering 10 numbers, iterates through list a, prints the number
#(print(i)) and adds it to the accumulator s (s += i).
print(f'La suma es {s}')
#imprime la suma total de los números guardados usando un f-string.
#Prints the total sum of the stored numbers using an f-string.
```

```
Escribe un numero: 9
Escribe un numero: 9
Escribe un numero: 3
Escribe un numero: 4
Escribe un numero: 5
Escribe un numero: 6
Escribe un numero: 7
Escribe un numero: 8
Escribe un numero: 9
Escribe un numero: 10
9
9
3
4
5
6
7
8
9
10
La suma es 70
```

PRACTICA 5

10 elementos, Números en arreglo, Caracteres en Listas y muestra cuantos elementos hay en cada estructura

```
'''Hacer un programa que lea 10 datos, Si el dato es un numero se almacenara en un arreglo si es un caracter o caracteres se metera en una lista, cuando finalice el programa mostrara cuantos elementos numericos y cuantos elementos hay en cada estructura'''
```

```
'''Make a program that reads 10 inputs.  
If the input is a number, it will be stored in an array.  
If it is a character or characters, it will be added to a list.  
When the program ends, it will show how many numeric elements and how many elements there are in each structure.'''
```

```
arr = [-1,-1,-1,-1,-1,-1,-1,-1,-1,-1]  
#Se crea un arreglo con 10 elementos -1 indica vacío o no asignado  
#Creates an array with 10 elements; -1 indicates empty or unassigned  
carc= []  
#Se crea una lista donde se guardara los caracteres  
#Creates a list where characters will be stored  
c = 0  
#Contador insertar en arr  
#Counter to insert into arr  
c2 = 0  
#Contador 2 contar valores numericos guardados en arr  
#Counter 2 to count numeric values stored in arr
```

```
while(True):  
#Ciclo infinito seguirá pidiendo datos hasta que se ejecute un break  
#Infinite loop that will keep asking for input until a break is executed  
    a = input('Escribe un dato: ')  
    #Pide al usuario que escriba un dato y lo guarda en variable a  
    #Asks the user to enter a value and stores it in variable a  
    if a.isdigit():  
        #Validar dígitos  
        #Validate digits  
        arr[c] = int(a)  
        #Si 'a' es todo dígitos lo convierte a entero con y lo guarda en arr en la posición c  
        #If 'a' is all digits, converts it to an integer and stores it in arr at position c  
    elif a.isalpha():  
        #Validar letras  
        #Validate letters  
        carc.append(a)  
        #Agrega elemento al final de la lista  
        #Adds the element to the end of the list  
    c += 1  
    #Cuenta las entradas que son 10 en total ya sea dígito o letra  
    #Counts the entries, which will total 10 whether digit or letter  
    if c >= 10:  
        break  
    #Si c llega a 10 (o más), sale del while con break  
    #If c reaches 10 (or more), exit the while loop with break
```

```
print(f'la lista tiene {len(carc)}')
#Muestra cuántos elementos textuales quedaron en la lista carc usando len(carc).
#Shows how many textual elements are in the list carc using len(carc)
for i in arr:
#Recorre i de arr
#Iterates through each element i in arr
    if i != -1:
        #Si i != -1 incrementa c2
        #If i != -1, increment c2
        c2 += 1
    #c2 será el conteo de elementos numéricos guardados en arr.
    #c2 will be the count of numeric elements stored in arr

print(f'el arreglo tiene {c2}')
```

```
#Imprime cuántos números se guardaron en arr
#Prints how many numbers were stored in arr
print(carc)
print(arr)
#Muestra por pantalla la lista carc (los textos) y la lista arr (con números).
#Displays the list carc (texts) and the list arr (with numbers)
```

```
Escribe un dato: louis
Escribe un dato: 45
Escribe un dato: 34
Escribe un dato: 35
Escribe un dato: tele
Escribe un dato: lapiz
Escribe un dato: sillón
Escribe un dato: 89
Escribe un dato: 7
Escribe un dato: 65
la lista tiene 4
el arreglo tiene 6
['louis', 'tele', 'lapiz', 'sillon']
[-1, 45, 34, 35, -1, -1, -1, 89, 7, 65]
```

PRACTICA 6

Pide lo mismo que la P5 pero con método principal

```
def resultados():
#Declara la función resultados
#Declares the function resultados
    c2 = 0
    print(f'la lista tiene {len(carc)}')
    for i in arr:
        if i != -1:
            c2 += 1
    print(f'el arreglo tiene {c2}')
    print(carc)
    print(arr)

def hola():
#Declara la funcion hola
#Declares the function hola
    c = 0
    while(True):
        #Ciclo infinito que se sale del ciclo con un while cuando se cumplen los 10 elementos
        #Infinite loop that exits when 10 elements are processed
        a = input('Escribe un dato: ')
        if a.isdigit():
            #VALIDAR DIGITOS (numeros)
            #VALIDATE DIGITS (numbers)
            arr[c] = int(a)
        elif a.isalpha():
            #VALIDAR LETRAS
            #VALIDATE LETTERS
            carc.append(a)
            #Agrega elemento al final de la lista
            #Adds element to the end of the list
            c += 1

        if c >=10:
            break
        resultados()
        #Llama a la función para imprimir el estado actual de carc y arr.
        #Calls the function to print the current state of carc and arr.
    carc= []
    arr =[-1,-1,-1,-1,-1,-1,-1,-1,-1,-1]

if __name__ == '__main__': #METODO PRINCIPAL #MAIN METHOD
    hola()
```



```
Escribe un dato: osiris
la lista tiene 1
el arreglo tiene 0
['osiris']
[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1]
Escribe un dato: 98
la lista tiene 1
el arreglo tiene 1
['osiris']
[-1, 98, -1, -1, -1, -1, -1, -1, -1, -1]
Escribe un dato: 90
la lista tiene 1
el arreglo tiene 2
['osiris']
[-1, 98, 90, -1, -1, -1, -1, -1, -1, -1]
Escribe un dato: 100
la lista tiene 1
el arreglo tiene 3
['osiris']
[-1, 98, 90, 100, -1, -1, -1, -1, -1, -1]
Escribe un dato: hola
la lista tiene 2
el arreglo tiene 3
['osiris', 'hola']
[-1, 98, 90, 100, -1, -1, -1, -1, -1, -1]
Escribe un dato: 1
la lista tiene 2
el arreglo tiene 4
['osiris', 'hola']
['osiris', 'hola']
[-1, 98, 90, 100, -1, 1, -1, -1, -1, -1]
Escribe un dato: 1000
la lista tiene 2
el arreglo tiene 5
['osiris', 'hola']
[-1, 98, 90, 100, -1, 1, 1000, -1, -1, -1]
Escribe un dato: Holaa
la lista tiene 3
el arreglo tiene 5
['osiris', 'hola', 'Holaa']
[-1, 98, 90, 100, -1, 1, 1000, -1, -1, -1]
Escribe un dato: Louis
la lista tiene 4
el arreglo tiene 5
['osiris', 'hola', 'Holaa', 'Louis']
[-1, 98, 90, 100, -1, 1, 1000, -1, -1, -1]
Escribe un dato: 10
```

PRACTICA 7

Crear una lista donde se almacenará 5 elementos pidiendo Nombre, Edad y Sexo

```
'''Hacer un programa que lea nombre, edad y sexo de 5 personas,
estos elementos tiene que estar dentro de una lista'''

'''Make a program that reads name, age, and gender of 5 people,
these elements must be inside a list'''

def inicio():
#Declara la función inicio
#Declares the function inicio
    co = 0
    #Contador en 0
    #Counter set to 0
    while True:
#Inicia un bucle infinito solo se sale del mismo con break
#Starts an infinite loop, only exits with break
        a = input('Introduce tu Nombre: ')
        b = input('Introduce tu Edad: ')
        c = input('Introduce tu Sexo: ')
        #Pide al usuario que agregue un dato y lo guarda en sus variables
        #Asks the user to add a value and stores it in its variables
        personas.append(a+b+c)
        #Añade a la lista global personas la concatenación de a, b y c
        #Adds to the global list personas the concatenation of a, b and c
        co += 1
        #Incrementa el contador co en 1
        #Increments the counter co by 1
        if co >= 5:
            break

#Comprueba si ya se han leído 5 entradas se cierra si llego a 5
#Checks if 5 entries have already been read, closes if it reached 5

print(personas)
#Imprime el contenido de la lista personas
#Prints the content of the list personas

personas = []
#Inicio de la lista llamada personas
#Initialization of the list called personas

if __name__ == '__main__':
#METODO PRINCIPAL
#MAIN METHOD
    inicio()
    #Llama la funcion para empezar a pedir los datos
    #Calls the function to start asking for the data
```

Introduce tu Nombre: osiris

Introduce tu Edad: 20

Introduce tu Sexo: F

Introduce tu Nombre: Luis

Introduce tu Edad: 23

Introduce tu Sexo: M

Introduce tu Nombre: Jose

Introduce tu Edad: 19

Introduce tu Sexo: M

Introduce tu Nombre: Luna

Introduce tu Edad: 18

Introduce tu Sexo: F

Introduce tu Nombre: Rosa

Introduce tu Edad: 30

Introduce tu Sexo: F

['osiris20F', 'Luis23M', 'Jose19M', 'Luna18F', 'Rosa30F']

PRACTICA 8

Leer una cadena y que muestre en pantalla cuantos números tiene, cuantas mayúsculas, minúsculas y espacios

```
'''Hacer un programa que lea una cadena y que muestre en pantalla cuantos numeros tiene
cuantas mayusculas, minusculas y espacios'''

'''Make a program that reads a string and displays on the screen how many numbers it has,
how many uppercase letters, lowercase letters, and spaces'''

def inicio():
#Declara la función inicio
#Declares the function inicio
    n = 0
    e = 0
    mi = 0
    may = 0
    #Inicializa contador de numeros, espacios,minusculas y mayusculas
    #Initializes counters for numbers, spaces, lowercase and uppercase
    numeros = "0123456789"
    #Variable que contiene los caracteres considerados como dígitos
    #Variable that contains the characters considered as digits

    cadena = input('Escribe una cadena: \n')
    #Pide al usuario que escriba una cadena
    #Asks the user to type a string
    for i in cadena:
        #Recorre la cadena carácter por carácter
        #Iterates through the string character by character

        if i in numeros:
            print('Es numero')
            n += 1
        #Si i es dígito imprime el numero e incrementa el contador
        #If i is a digit, prints the message and increments the counter
        if ord(i) == 32:
            e += 1
        #Comprueba si el código ASCII del carácter es 32 (Espacio) incrementa en contador
        #Checks if the ASCII code of the character is 32 (Space) and increments the counter
        if ord(i) >= 97 and ord(i) <= 122:
            mi += 1
        #Comprueba si ord(i) está entre letras minúsculas incrementa en contador
        #Checks if ord(i) is within lowercase letters and increments the counter
        if ord(i) >= 65 and ord(i) <= 90:
            may += 1
        #Comprueba si ord(i) está entre letras mayusculas incrementa en contador
        #Checks if ord(i) is within uppercase letters and increments the counter

    print(f'\n Los numeros son: {n}\n Los espacios son: {e}\n Las minusculas son: {mi}\n Las mayusculas son: {may}')
    #Imprime el resultado final usando un f-string
    #Prints the final result using an f-string

if __name__ == '__main__':
#Metodo Principal
#Main Method
    inicio()
```

```
Escribe una cadena:  
Osiris 10  
Es numero  
Es numero  
  
Los numeros son: 2  
Los espacios son: 1  
Las minusculas son: 5  
Las mayusculas son: 1
```

PRACTICA 9 TAREA

Crear una lista, no contener espacio, mayúscula 1ra letra, contener todas las vocales. 5 elementos

```
'''Hacer un programa que en una lista se introduzca cadenas de caracteres con las siguientes restricciones
```

- 1- Las cadenas no deben tener espacios
- 2- La cadena solo puede tener mayuscula la primer letra
- 3- Obligatoriamentes tiene que tener todas las vocales (a,e,i,o,u)

```
El programa no termina hasta que la lista tenga 5 elementos'''
```

```
'''Make a program that stores character strings in a list with the following restrictions:
```

- 1- The strings must not contain spaces
- 2- The string can only have the first letter in uppercase
- 3- It must necessarily contain all vowels (a, e, i, o, u)

```
The program does not end until the list has 5 elements'''
```

```
def inicio():  
#Declara la función inicio()  
##Declares the function inicio()  
    c = 0  
    #Inicializa un contador c en 0  
    ##Initializes a counter c to 0  
    vocales = set("aeiou")  
    #Crea un set con las vocales mínimas para usar en la comprobacion vocales  
    ##Creates a set with the vowels to use in the vowel check
```

```
while True:
    #Empieza un bucle infinito; la salida se consigue con break.
    #Starts an infinite loop; the exit is achieved with break.
    cadena = input('Escribe una cadena: \n')
    #Pide al usuario que escriba una cadena y lo guarda en variable cadena
    #Asks the user to enter a string and stores it in variable cadena
    if " " in cadena:
        #Comprueba si hay un espacio en la cadena.
        #Checks if there is a space in the string.
        print('No debe contener espacios\n')
        #Si la cadena contiene espacios, muestra un mensaje de error.
        #If the string contains spaces, shows an error message.
    if not cadena[0].isupper():
        print('La primera letra debe ser mayuscula')
        #Comprueba si el primer carácter no es mayúscula e imprime mensaje
        #Checks if the first character is not uppercase and prints a message
    if not vocales.issubset(cadena.lower()):
        print('Debe contener todas las vocales')
        #Convierte la cadena a minúsculas y verifica si el conjunto vocales está incluido en los caracteres de la cadena
        #Converts the string to lowercase and checks if the set of vowels is included in the characters of the string
    #CONTADOR DE ELEMENTOS / ELEMENTS COUNTER
    c += 1
    #Incrementa el contador c siempre, aunque la entrada haya sido inválida.
    #Increments the counter c always, even if the entry was invalid.
    #ELEMENTOS QUE QUIERO TENER / NUMBER OF ELEMENTS I WANT
    if c > 5:
        break
    #Comprueba si ya se han hecho 5 entradas si lo esta cierra con break

    #Checks if 5 entries have already been made; if so, closes with break
    lista.append(cadena)
    #Añade la cadena a la lista global lista
    #Adds the string to the global list called lista
    print(lista)
    #Imprime la lista
    #Prints the list

lista = []
#Inicializa la lista global llamada lista
#Initializes the global list named lista
if __name__=='__main__': #METODO PRINCIPAL / MAIN METHOD
    inicio()
    #Llama a la función inicio() para iniciar el programa
    #Calls the function inicio() to start the program
```

Escribe una cadena:

hola

La primera letra debe ser mayuscula

Debe contener todas las vocales

Escribe una cadena:

Holaeiu

Escribe una cadena:

Aeiou

Escribe una cadena:

Valentina

No debe contener espacios

Debe contener todas las vocales

Escribe una cadena:

Aeious

Escribe una cadena:

Aeiou

['hola', 'Holaeiu', 'Aeiou', 'Valentina ', 'Aeious']

PRACTICA 10

Pide lo mismo que la P9 pero otra solución agregando Validaciones

```
def vocales(cad):
#Declara la función vocales que recibe un parámetro cad
#Declares the function vocales that receives a parameter cad
    ba = False
    be = False
    bi = False
    bo = False
    bu = False
    #Inicializa la variable booleana en False
    #Initializes the boolean variables as False
    '''for i in cad:'''
    if 'a' in cad or 'A' in cad:
        ba = True
    if 'e' in cad or 'E' in cad:
        be = True
    if 'i' in cad or 'I' in cad:
        bi = True
    if 'o' in cad or 'O' in cad:
        bo = True
    if 'u' in cad or 'U' in cad:
        bu = True
    #Está en la cadena cad, marca ba,be,bi,bo,bu = True.
    #If the vowels are in the string cad, mark ba, be, bi, bo, bu as True.
    if ba == True and be == True and bi == True and bo == True and bu == True:
    #Comprueba si todas las variables son true
    #Checks if all the variables are True
        lista.append(cad)
        #Si la condición anterior es True, añade la cadena cad a la lista global
        #If the previous condition is True, add the string cad to the global list
print(lista)
#Imprime la lista "lista" / Prints the list "lista"

def minusculas(c1):
#Declara la función minusculas que recibe c1
#Declares the function minusculas that receives c1
    cm = 0
    #Contar caracteres minúsculos a partir de la segunda letra.
    #Count lowercase characters starting from the second letter.
    print(c1)
    #Imprime la cadena recibida / Prints the received string
    for i in c1[1:]:
    #Recorre la cadena desde el segundo carácter en adelante comprobar sea minusculas
    #Iterates through the string starting from the second character to check if they are lowercase
        if ord(i) >=97 and ord(i) <=122:
        #Comprueba si el código ASCII de minusculas
        #Checks if the ASCII code corresponds to lowercase
            cm += 1
            #Incrementa / Increments
    if cm == len(c1)-1:
        #Si todas las posiciones después de la primera se han contado como minúsculas
        #If all positions after the first have been counted as lowercase
        print(f'La cadena son minusculas excepto la primera{cm}')
        vocales(c1)
        #Llama a la función vocales PASÁNDOLE cm
        #Calls the function vocales PASSING cm
```

```
else:
    print('Error en la cadena')

def leer():
    #Declara la función leer / Declares the function leer
    con = 0 #Contar caracteres / Counts characters
    nc = "" #Nueva cadena como vacia / New string as empty
    c = input('Escribe una cadena \n')
    #Introducir una cadena / Prompt the user to enter a string
    for i in c:
        if ord(i) != 32:
            con += 1
        #Comprueba el espacio, si i no es espacio incrementa 'con'
        #Checks for spaces, if i is not a space increment 'con'
    if con == len(c):
        #Comprueba si con es igual a la longitud total de la cadena c
        #Checks if con is equal to the total length of the string c
        if c.isalpha():
            #Reviso que sean minusculas / Check that they are letters only
            minusculas(c)
        else:
            for i in c:
                if ord(i) >= 48 and ord(i) <= 57:
                    pass
                #Comprueba si el carácter es un numero, si es dígito no hace nada
                #Checks if the character is a number, if it is a digit does nothing
            else:
                nc += i
            nc += i
        #Quita los dígitos y copia el resto en nc.
        #Removes digits and copies the rest into nc.
        print(nc) #Imprime la cadena nc / Prints the string nc
        minusculas(nc)
        #Llama a minusculas con esa cadena sin dígitos
        #Calls minusculas with the string without digits
    else:
        print('Error en la cadena')

lista = [] #Creacion de la lista / Creation of the list
if __name__ == '__main__': #METODO PRINCIPAL / MAIN METHOD
    while(True): #Bucle infinito / Infinite loop
        leer()
        #LLama la funcion leer para pedir cadena y procesarla
        #Calls the function leer to request and process a string
        if len(lista) >= 5:
            #Comprueba si la lista ya tiene 5 elementos
            #Checks if the list already has 5 elements
            break
```

Escribe una cadena

Osirisaeiou

Osirisaeiou

La cadena son minusculas excepto la primera10

['Osirisaeiou', 'Osirisaeiou']

Escribe una cadena

Osirisaeiou

Osirisaeiou

La cadena son minusculas excepto la primera10

['Osirisaeiou', 'Osirisaeiou', 'Osirisaeiou']

Escribe una cadena

Louisaeiou

Louisaeiou

La cadena son minusculas excepto la primera9

['Osirisaeiou', 'Osirisaeiou', 'Osirisaeiou', 'Louisaeiou']

Escribe una cadena

Louisaeiou

Louisaeiou

La cadena son minusculas excepto la primera9

['Osirisaeiou', 'Osirisaeiou', 'Osirisaeiou', 'Louisaeiou', 'Louisaeiou']

PRACTICA 11 REPASO

Almacenar Nombre, Precio de producto y calcular el costo (12% y 16%) y precio de venta

Hacer un programa que lea

1.- Nombre

2.- Precio de un producto

el programa calculara el costo y precio de venta

costo involucra (12%) y iva (16%)'''

`while(True):`

`#Ciclo infinito que termina con un break si el usuario desea salir`

`#Infinite loop that ends with a break if the user wants to exit`

`Nom = input('Ingresa tu nombre: ')`

`#Usuario ingresa Nombre y guarda en variable 'Nom'`

`#User enters name and stores it in variable 'name'`

`Pre = float(input('Ingresa el precio del producto: '))`

`#Ingresa Precio en Float = Decimal`

`#Enter price as float = decimal`

`costo = Pre * 1.12`

`#Variable costo que hara operacion multiplicando Pre con el 12%`

`#Variable cost will do the operation multiplying price by 12%`

`preciov = Pre * 1.16`

`#Variable preciov hace operacion Pre multiplicando el 16%`

`#Variable sale_price does the operation multiplying price by 16%`

`print(f'El costo es: {costo:.2f} \nEl precio de venta es: {preciov:.2f}')`

`#La consola mostrara el mensaje con resultado d elas operaciones`

`#The console will show the message with the result of the operations`

```
respuesta = input('Deseas otro numero (s/n): \n')
#Variable respuesta que pide al usuario si desea seguir o no
#Variable answer asks the user if they want to continue or not
if respuesta == 'n' or respuesta == 'N':
#Las respuestas que solo puede dar el usuario al salir
#Valid responses to exit the program
    break
#Cierra el programa cuando el usuario ingrese n o N
#Closes the program when the user enters n or N
```

Ingresa tu nombre: Shampoo

Ingresa el precio del producto: 75

El costo es: 84.00

El precio de venta es: 87.00

Deseas otro numero (s/n):

n

PS C:\Users\T470\Documents\Practicas python\Practicas Parcial 1>

PRACTICA 12 REPASO

Formula General

```
'''Formula General'''  
'''Quadratic Formula'''  
a = 1  
b = 2  
c = -15  
#Asignamos valores a (a,b y c) para hacer la operacion  
#Assign values to (a, b, and c) to perform the operation  
  
p = 0 #Potencia / Power  
m = 0 #Multiplicacion / Multiplication  
r = 0 #Resta / Subtraction  
ra =0.0 #Raiz / Square root  
d = 0.0 #Denominador / Denominator  
x1 = 0.0 #1er Resultado / 1st result  
x2 = 0.0 #2do Resultado / 2nd result  
#Variables de las operaciones a realizar  
#Variables for the operations to be performed
```

```
p = b**2 #Potencia b = 2 ** 2 Fijo
m = 4*a*c #Multi 4 fijo *1*-15
r = p - m #Resta Potencia - Multi
if r > 0:
    #Comprueba si el resultado de r es positiva.
    #Checks if the result of r is positive
    print('Si se puede')
    #Mensaje si es positivo sigue con el programa
    #Message if positive, continue with the program
    ra = r**(1/2)
    #Calcula la Raiz cuadrada
    #Calculates the square root
    d = 2*a
    #Calcula Denominador multiplicando 2 fijo * 1
    #Calculates denominator multiplying 2 fixed * 1
    x1 = (-b + ra)/d
    #Resultado positivo
    #Positive result
    x2 = (-b - ra)/d
    #Resultado Negativo
    #Negative result
    print(f'El valor de X1 es {x1:.2f} El valor de X2 es {x2:.2f}')
    #Muestra los resultados
    #Displays the results
else:
    print('No se puede')
    #Si el resultado r fuera negativa terminara el programa
    #If the result r were negative, the program would end
```

Si se puede

El valor de X1 es 3.00 El valor de X2 es -5.00

PRACTICA 13 REPASO

Leer dato cualquiera y almacenarlo en Lista (Entero y Decimal) con try y ValueError

```
'''Hacer un programa que lea un dato (cualquiera) y que lo almacene en una lista respetando
su tipo de dato '''
'''Make a program that reads any data entered by the user
and stores it in a list while respecting its data type.'''

def validar(d):
    #Define la función validar que recibe un parámetro d
    #Define the function validate that receives a parameter d
    ne = 0 #Numero Entero / Integer number
    dec = 0.0 #Numero Decimal / Decimal number
    |
    '''ENTERO'''
    try:
        #Bloque try para intentar la conversión a entero
        #Try block to attempt conversion to integer
        ne = int(d)
        #Convertir d a un entero / Convert d to an integer
        print("Es un entero")
        print("No es un decimal")
        #Mensajes / Messages
        return ne
        #Si la conversión a entero tuvo éxito, la función devuelve ese entero y termina
        #If the conversion to int was successful, return that integer and end the function
    except ValueError:
        print('No es un entero')
        #Si int(d) falla se captura ValueError
        #If int(d) fails, ValueError is caught
```



```
'''DECIMAL'''

try:
    #bloque try para intentar la conversión a decimal
    #If int(d) fails, ValueError is caught
    dec = float(d)
    #Convertir d a decimal / Convert d to float
    print("Es un decimal")
    #Mensaje / Message
    return dec
    #Si la conversión a float tiene éxito, devuelve ese número float y termina la función.
    #If the conversion to float was successful, return that float and end the function
except ValueError:
    print('No es un decimal')
    #Si float(d) falla se ejecuta ValueError
    #If float(d) fails, ValueError is executed

def leer():
    #Definir funcion leer / Define function read
    d = input('Ingresa un dato cualquiera: ')
    #Ingresa dato el usuario y almacena variable d
    #User enters data and it is stored in variable d
    dato = validar (d)
    #Llama función validar con d. validar intentará convertirlo a int o float
    #Calls validate function with d. validate will try to convert it to int or float
    lista.append(dato)
    #Añade (append) el dato ya validado a la lista llamada lista
    #Adds (append) the validated data to the list called mylist

lista = []
#Creacion de la lista / Create the list

if __name__ == '__main__': #METODO PRINCIPAL / MAIN METHOD
    while(True): #incia un bucle infinito / Starts an infinite loop
        leer()
        #LLama la funcion para pedir dato y agregarlo a la lista
        #Calls the function to request data and add it to the list
        respuesta = input('Deseas otro (s/n): \n')
        if respuesta == 'n' or respuesta == 'N':
            #Pregunta al usuario si desea ingresar otro dato; captura su respuesta en respuesta
            #Asks the user if they want to enter another piece of data; stores the answer in answer
            print(lista)
            #Si el usuario eligió salir, imprime la lista completa con todos los datos ingresados
            #If the user chose to exit, print the full list with all entered data
            break
        #Rompe bucle while y termina el programa
```

Ingresa un dato cualquiera: 3

Es un entero

No es un decimal

Deseas otro (s/n):

s

Ingresa un dato cualquiera: 3.3

No es un entero

Es un decimal

Deseas otro (s/n):

n

[3, 3.3]

PARCIAL 2

PRACTICA 1

Pide 5 calificaciones y agrega a una lista

```
'''Pide al usuario 5 Calificaciones y se agregan a una lista'''

def inicio(num):
    #Definimos la función 'inicio' que recibe un parámetro llamado 'num' (un contador)
    #Define the function 'inicio' which takes one parameter called 'num' (a counter)
    #global num
    a = int(input('Escribe una calificacion \n'))
    #Pide al usuario que escriba una calificación.
    #'input()' devuelve texto, por eso se convierte a entero con int().
    #Asks the user to enter a grade.
    #'input()' returns text, so it's converted to an integer with int().

    num += 1
    #Incrementa el contador 'num' en 1 (lleva la cuenta de cuántas calificaciones se han ingresado)
    #Increases the counter 'num' by 1 (keeps track of how many grades have been entered)
    lista.append(a)
    #Agrega la calificación ingresada a la lista global 'lista'
    #Adds the entered grade to the global list 'lista'
    if num >= 5:
        #Si ya se han ingresado 5 calificaciones o más
        #If 5 or more grades have been entered
        print(lista)
        #Entonces imprime la lista completa de calificaciones.
        #Then print the complete list of grades.
    else:
        #Si todavía no se han ingresado 5 calificaciones.
        #If fewer than 5 grades have been entered.

        inicio(num)
        #Vuelve a llamar a la función (recursión) para pedir otra calificación.
        #Call the function again (recursion) to ask for another grade.

a = []
a una lista vacía en la que se guardarán las calificaciones
ates an empty list where the grades will be stored
al num
a línea fuera de cualquier función no es necesaria, pero indica que 'num' será una variable glob
s line outside any function is unnecessary, but it indicates that 'num' will be a global variabl
= 0
inicializa la variable 'num' con 0 (contador de calificaciones)
tializes the variable 'num' to 0 (the grade counter)
__name__=='__main__':
inicio(num)
```

```
Escribe una calificacion
9
Escribe una calificacion
8
Escribe una calificacion
7
Escribe una calificacion
6
Escribe una calificacion
5
[9, 8, 7, 6, 5]
```

PRACTICA 2

Pide 5 calificaciones y al final se muestra la lista de las calificaciones y la suma total del promedio general, valida los números en otro archivo

ARC 1

```
2  '''Ingresamos 5 Calificaciones validadas en Numeros llamando a otro archivo,
3  si ingresas otro digito que no sea un numero marca error por lo que tienes que
4  ingresar otra calificacion (recursividad) hasta que cumplas con los 5 elementos,
5  al final se almacenara en una lista y se mostrara en la consola al terminar lo 5 elementos'''
6
7  from validaciones import validacion
8  #Importamos la clase 'validacion' del archivo 'validaciones.py'
9  # Import the class 'validacion' from the file 'validaciones.py'
10 | | #Archivo          #Clase
11 val = validacion()
12 #Creamos un objeto (instancia) de la clase 'validacion' para usar sus métodos
13 #Create an object (instance) from the class 'validacion' so we can use its methods
14 class Principal():
15 #Definimos una clase llamada 'Principal'
16 #Define a class called 'Principal'
17
18 def __init__(self):
19 #Método constructor, se ejecuta automáticamente al crear un objeto
20 #Constructor method, runs automatically when creating an object
21     self.lista = []
22     #Creamos una lista vacía donde se guardarán las calificaciones
23     #Create an empty list to store the grades
24     self.num = 0
25     #Inicializamos un contador para saber cuántas calificaciones válidas se ingresaron
26     #Initialize a counter to track how many valid grades were entered
27     self.a = ""
28     #Variable temporal para almacenar lo que el usuario escriba (como texto)
29     #Variable to temporarily store the user input (as a string)
```

```
31 def inicio(self):
32     #Definimos el método 'inicio', encargado del flujo principal del programa
33     #Define a method called 'inicio' (start) that runs the main logic
34     self.a = input('Escribe una calificacion \n')
35     #Pedimos al usuario que escriba una calificación
36     #Ask the user to input a grade and store it in 'self.a'
37     if val.validarNumeros(self.a):
38         #Usamos el método 'validarNumeros' del objeto 'val' para verificar si es un número
39         #Use the method 'validarNumeros' from the 'val' object to check if input is a number
40         self.num += 1
41         #Si la calificación es válida, aumentamos el contador en 1
42         #Increase the counter by 1 since the input was valid
43         self.lista.append(int(self.a))
44         #Convertimos la entrada a entero y la añadimos a la lista de calificaciones
45         #Convert the input to an integer and add it to the list of grades
46         if self.num >= 5:
47             #Si ya se ingresaron 5 o más calificaciones válidas
48             #If 5 or more valid grades have been entered
49             print(self.lista)
50             #Mostramos la lista completa de calificaciones
51             #Print the complete list of grades
52             print(f'El promedio es: {val.Promedio(self.lista)}')
53             #Calculamos e imprimimos el promedio usando el método 'Promedio'
54             #Calculate and print the average using 'Promedio' method
55         else:
56             self.inicio()
57             #Si aún no se completan las 5 calificaciones, volvemos a llamar al método (recursividad)
58             #If less than 5 grades, call 'inicio' again (recursion) to keep asking
59         else:
60             print('No es un numero')
61             #Si la entrada no es un número, mostramos un mensaje de error
62             #Show message if the input is not a valid number
63             self.inicio()
64             #Volvemos a llamar al método 'inicio' hasta que se ingrese un valor válido
65             #Call 'inicio' again (recursion) to ask again until a valid number is entered
66
67 if __name__ == '__main__':
68     app = Principal()
69     #Creamos un objeto llamado 'app' de la clase 'Principal'
70     #Create an object 'app' from the 'Principal' class
71     app.inicio()
72     #Llamamos al método 'inicio' para iniciar el proceso de ingreso de calificaciones
73     #Call the 'inicio' method to start the program and begin asking for grades
```

ARC 2

```
3 class validacion():
4     #Definimos una clase llamada 'validacion'
5     # Define a class named 'validacion'
6
7     def __init__(self):
8         #Método constructor: se ejecuta automáticamente al crear un objeto de esta clase
9         #Constructor method: runs automatically when creating an object of this class
10        self.suma = 0
11        #Variable que guardará la suma total de las calificaciones
12        #Variable that stores the total sum of the grades
13        self.promedio = 0.0
14        #Variable que guardará el promedio (como número decimal)
15        #Variable that stores the average (as a decimal number)
16
17    def validarNumeros(self, valor):
18        #Método para validar si un valor ingresado es numérico
19        #Method to validate if the entered value is numeric
20        if valor.isdigit():
21            #El método .isdigit() devuelve True si todos los caracteres del texto son números
22            # The .isdigit() method returns True if all characters in the string are digits
23            return True
24            #Si es un número, regresa True (válido)
25            #If it is a number, return True (valid)
26        else:
27            return False
28            #Si no es un número, regresa False (inválido)
29            #If it is not a number, return False (invalid)
30
31    def Promedio(self, lista):
32        #Método para calcular el promedio de los valores dentro de una lista
33        #Method to calculate the average of the values in a list
34        for i in lista:
35            #Recorremos cada elemento 'i' dentro de la lista de calificaciones
36            # Loop through each element 'i' inside the list of grades
37            self.suma += i
38            #Sumamos cada calificación al acumulador 'suma'
39            #Add each grade to the accumulator 'suma'
40        self.promedio = self.suma / len(lista)
41        #Calculamos el promedio dividiendo la suma entre el total de elementos
42        #Calculate the average (sum divided by number of elements)
43        return self.promedio
44        #Devolvemos el promedio calculado
45        #Return the calculated average
```


Escribe una calificacion

7

Escribe una calificacion

6

Escribe una calificacion

8

Escribe una calificacion

7

Escribe una calificacion

6

[7, 6, 8, 7, 6]

El promedio es: 6.8

PRACTICA 3

Pide usuario y contraseña mediante cajas de texto, label y botón con mensajes de error y correcto

```
'''Creamos una ventana donde se le pide al usuarios ingresar USUARIO y CONTRASEÑA  
valida si coinciden con la informacion fija admin y 12345 seguido de esto,  
muestra un mensaje si es correcto o error segun el caso'''
```

```
from tkinter import *  
#Importa todas las clases y funciones de Tkinter  
#Imports all classes and functions from Tkinter  
from tkinter import messagebox  
#Importa el módulo 'messagebox' para diálogos (info/error)  
#Imports the 'messagebox' module for dialog boxes (info/error)
```

```
def Ventana():  
    #Define la función principal que crea la ventana  
    #Defines the main function that creates the window  
  
    def revisar():  
        #Define una función interna que se ejecuta al pulsar el botón  
        #Defines an inner function that runs when the button is clicked  
        try:  
            u = str(us.get())  
            #Obtiene el texto del Entry 'us' (usuario). .get() ya devuelve str  
            #Gets the text from the 'us' Entry (username). .get() already returns str  
            p = str(pas.get())  
            #Obtiene el texto del Entry 'pas' (contraseña).  
            #Gets the text from the 'pas' Entry (password).  
            if u == 'admin' and p == '12345':  
                #Comprueba las credenciales hardcodeadas: usuario 'admin' y contraseña '12345'  
                #Checks hardcoded credentials: username 'admin' and password '12345'  
                messagebox.showinfo('Validacion','Usuario y Contraseña correcta')  
                #Si coinciden, muestra un cuadro informativo con título y mensaje  
                #If they match, display an info box with a title and message  
            else:  
                messagebox.showerror('Error','Usuario y/o Contraseña incorrecta')  
                #Si no coinciden, muestra un cuadro de error con título y mensaje  
                #If they don't match, display an error box with title and message  
  
        except ValueError:  
            messagebox.showerror('Error','Introduce datos')  
            #Muestra un cuadro de error solicitando introducir datos válidos.  
            #Shows an error box asking the user to enter valid data.
```

```
ven = Tk()
#Crea la ventana principal (instancia de Tk)
#Creates the main window (instance of Tk)
ven.title('Programa 1 con Ventanas')
#Establece el título de la ventana
#Sets the title of the window
ven.geometry('400x400')
#Define el tamaño de la ventana: 400x400 píxeles
#Defines the size of the window: 400x400 pixels

Label(ven, text='Usuario').pack(pady=10)
#Crea una etiqueta con el texto 'Usuario' dentro de 'ven' y la empaqueta (pack) con separación vertical (pady)
#Creates a label with the text 'Username' inside 'ven' and packs it with vertical spacing (pady)
us = Entry(ven)
#Crea una caja de texto (Entry) para el usuario dentro de la ventana 'ven'
#Creates a text box (Entry) for the username inside the window 'ven'
us.pack(pady=3)
#Empaqueta la caja de texto y agrega un pequeño espaciado vertical
#Packs the text box and adds small vertical spacing

Label(ven, text='Contraseña').pack(pady=10)
#Crea otra etiqueta para 'Contraseña' y la empaqueta con separación
#Creates another label for 'Password' and packs it with spacing
pas = Entry(ven)
#Crea una caja de texto para la contraseña
#Creates a text box for the password
pas.pack(pady=3)
#Empaqueta la caja de la contraseña
#Packs the password text box

boton = Button(ven, text='Aceptar', command=revisar).pack(pady=3)
#Crea un botón que llama a la función 'revisar' cuando se pulsa.
#Creates a button that calls the 'revisar' function when pressed.

ven.mainloop()
#Inicia el bucle principal de la ventana (mantiene la ventana abierta y responsive)
#Starts the main window loop (keeps it open and responsive)

if __name__ == '__main__':
    Ventana()
    #Llama a la función 'Ventana' para mostrar la interfaz
    #Calls the 'Ventana' function to display the interface
```

Programa 1 con Ventanas

Usuario

admin

Contraseña

12345

Aceptar

Validacion

Usuario y Contraseña correcta

Aceptar

Programa 1 con Ventanas

Usuario

admin

Contraseña

1234

Aceptar

Error

Usuario y/o Contraseña incorrecta

Aceptar

PRACTICA 4

Pide usuario y contraseña mediante cajas de texto, label y botón con mensajes de error y correcto con clases

```
'''Crear una ventana con dos label, dos cajas de texto y un boton  
que ingresaras el usuario y contraseña que al presionar el boton  
mostrara si los datos cson correctos o incorrectos CLASE'''
```

```
from tkinter import *  
#Importa todas las clases y funciones de Tkinter  
#Imports all classes and functions from Tkinter  
from tkinter import messagebox  
#Importa el módulo 'messagebox' para mostrar cuadros de diálogo  
#Imports the 'messagebox' module to display dialog boxes  
  
class Ventana():  
#Define una clase llamada 'Ventana'  
#Defines a class named 'Ventana'  
    def __init__(self):  
        #Método constructor: se ejecuta al crear una instancia de la clase  
        #Constructor method: runs automatically when creating a class instance  
        self.ven = Tk()  
        #Crea la ventana principal usando Tk()  
        #Creates the main window using Tk()  
        self.ven.title('Programa 1 con Ventanas')  
        #Establece el título de la ventana  
        #Sets the title of the window  
        self.ven.geometry('400x400')  
        #Define el tamaño de la ventana (400x400 píxeles)  
        #Defines the window size (400x400 pixels)  
        self.inicio()  
        #Llama al método 'inicio' para construir la interfaz  
        #Calls the 'inicio' method to build the interface
```

```
def inicio(self):
#Método que crea los elementos (widgets) de la interfaz
#Method that creates the interface elements (widgets)
    Label(self.ven, text='Usuario').pack(pady=10)
#Crea una etiqueta con el texto 'Usuario' y la muestra
#Creates a label with the text 'Usuario' and displays it

    self.us = Entry(self.ven)
#Crea una caja de texto para ingresar el usuario
#Creates a text box to enter the username
    self.us.pack(pady=3)
#Muestra la caja con un espacio vertical pequeño
#Displays the text box with small vertical spacing

    Label(self.ven, text='Contraseña').pack(pady=10)
#Crea una etiqueta para la contraseña
#Creates a label for the password

    self.pas = Entry(self.ven)
#Crea una caja de texto para ingresar la contraseña
#Creates a text box to enter the password
    self.pas.pack(pady=3)
#Muestra la caja con un espacio pequeño
#Displays the box with small spacing
```

```
boton = Button(self.ven,text='Aceptar',command=self.revisar).pack(pady=3)
#Crea un botón con el texto 'Aceptar' que ejecuta 'self.revisar' al presionarlo
#Creates a button labeled 'Aceptar' that executes 'self.revisar' when clicked

self.ven.mainloop()
#Inicia el bucle principal de la ventana (mantiene la ventana abierta)
#Starts the main event loop (keeps the window open)

def revisar(self):
#Método que valida el usuario y la contraseña
#Method that validates username and password
    try:
        u = str(self.us.get())
        #Obtiene el texto de la caja 'us' (usuario)
        #Gets the text from the 'us' (username) entry box
        p = str(self.pas.get())
        #Obtiene el texto de la caja 'pas' (contraseña)
        #Gets the text from the 'pas' (password) entry box

        if u == 'admin' and p == '12345':
            #Compara si usuario y contraseña son correctos
            #Checks if username and password are correct
            messagebox.showinfo('Validacion','Usuario y Contraseña correcta')
            #Muestra mensaje de éxito
            #Displays success message
        else:
            messagebox.showerror('Error','Usuario y/o contraseña incorrecta')
            #Muestra mensaje de error si los datos no coinciden
            #Displays error message if credentials don't match
    except ValueError:
        messagebox.showerror('Error','Introduce datos')
        #Muestra mensaje de error si ocurre un problema al leer los datos
        #Displays error message if a data reading issue occurs

if __name__=='__main__':
    Ventana()
#Crea una instancia de la clase Ventana y muestra la interfaz
#Creates an instance of the Ventana class and displays the interface
```

Programa 1 con Ventanas

Usuario

admin

Contraseña

12345

Aceptar

Validacion

Usuario y Contraseña correcta

Aceptar

Programa 1 con Ventanas

Usuario

admin

Contraseña

1234

Aceptar

Error

Usuario y/o contraseña incorrecta

Aceptar

PRACTICA 5

3 Calificaciones con lista, orden descendente

```
'''Hacer un programa que lea
-Nombre
-3 Calificaciones
que se agregaran a una lista con las siguientes restricciones
-Calificaciones se agregaran en orden descendente
-Mostrara una pregunta si deseas agregar otra persona, si la respuesta es correcta
-La primera lista se agregara a otra lista que contenga los 4 datos para agregar
otra persona'''
```

```
def inicio():
#Define la función 'inicio' que procesa una persona y sus 3 calificaciones
#Defines the 'inicio' function that processes one person and their 3 grades.
    lista1 = []
    #Lista temporal para guardar nombre y calificaciones ordenadas.
    #Temporary list to store name and ordered grades.
    nom = input('Escribe tu nombre: ')
    #Pide el nombre de la persona.
    #Asks for the person's name.

    cal1 = int(input('Agrega tu primera Calificacion: '))
    #Pide la primera calificación y la convierte a entero.
    #Asks for the first grade and converts it to int.
    cal2 = int(input('Agrega tu segunda Calificacion: '))
    #Pide la segunda calificación y la convierte a entero.
    #Asks for the second grade and converts it to int.
    cal3 = int(input('Agrega tu tercera Calificacion: '))
    #Pide la tercera calificación y la convierte a entero.
    #Asks for the third grade and converts it to int.

    #Comparamos las calificaciones para determinar mayor, medio y menor.
    #Compare the grades to determine largest, middle and smallest.
    if(cal1 > cal2):
    #Si cal1 es mayor que cal2.
    #If cal1 is greater than cal2.
        if(cal1 > cal3):
            #cal1 también es mayor que cal3 (cal1 es la mayor).
            #And cal1 is also greater than cal3 (cal1 is the largest).
            print(f'Es mayor {cal1}')
            #Imprime cuál es la mayor.
            #Prints which one is the largest.
            lista1.append(nom)
            #Añade el nombre a la lista temporal.
            #Adds the name to the temporary list.
```

```
lista2.append(cal1)
#Añade directamente la mayor a lista2 (inconsistencia con el resto).
#Appends the largest directly to lista2 (inconsistent with other branches).
if(cal2 > cal3):
    #Si cal2 es mayor que cal3, entonces cal2 es la del medio y cal3 la menor.
    #If cal2 is greater than cal3, then cal2 is middle and cal3 is smallest.
    print(f'Es el de en medio{cal2}')
    #Imprime la calificación del medio.
    #Prints the middle grade.
    print(f'Es el menor{cal3}')
    #Imprime la calificación menor.
    #Prints the smallest grade.
    lista1.append(cal2)
    #Añade cal2 (medio) a la lista temporal.
    #Adds cal2 (middle) to the temporary list.
    lista1.append(cal3)
    #Añade cal3 (menor) a la lista temporal.
    #Adds cal3 (smallest) to the temporary list.
    lista2.append(lista1)
    #Añade la lista temporal completa a la lista global lista2.
    #Appends the full temporary list to the global lista2.
```

```
else:
    #Si cal3 >= cal2, entonces cal3 es la del medio y cal2 la menor.
    #If cal3 >= cal2, then cal3 is middle and cal2 is smallest.
    print(f'Es el de en medio{cal3}')
    #Imprime la calificación del medio.
    #Prints the middle grade.
    print(f'Es el menor {cal2}')
    #Imprime la calificación menor.
    #Prints the smallest grade.

    lista1.append(cal3)
    #Añade cal3 (medio) a la lista temporal.
    #Adds cal3 (middle) to the temporary list.
    lista1.append(cal2)
    #Añade cal2 (menor) a la lista temporal.
    #Adds cal2 (smallest) to the temporary list.
    lista2.append(lista1)
    #Añade la lista temporal completa a lista2.
    #Appends the full temporary list to lista2.
```

```
#Si cal1 > cal2 pero cal1 <= cal3, entonces cal3 es la mayor.  
#If cal1 > cal2 but cal1 <= cal3, then cal3 is the largest.  
else:
```

```
    print(f'Es el de en medio {cal1}')
```

#Aquí cal1 sería la del medio.
#Here cal1 would be the middle grade.

```
    print(f'Es mayor {cal3}')
```

#Imprime la mayor (cal3).
#Prints the largest (cal3).

```
    print(f'Es el menor {cal2}')
```

#Imprime la menor (cal2).
#Prints the smallest (cal2).

```
lista1.append(nom)  
#Añade el nombre a la lista temporal.  
#Adds the name to the temporary list.  
lista1.append(cal3)  
#Añade la mayor (cal3) primero.  
#Adds the largest (cal3) first.  
lista1.append(cal1)  
#Añade la del medio (cal1).  
#Adds the middle grade (cal1).  
lista1.append(cal2)  
#Añade la menor (cal2).  
#Adds the smallest (cal2).  
lista2.append(lista1)  
#Añade la lista temporal ordenada a lista2.  
#Appends the ordered temporary list to lista2.
```

```
#Caso en que cal1 <= cal2 (cal2 podría ser mayor que cal1).
#Case where cal1 <= cal2 (cal2 might be greater than cal1).
else:
    if(cal2 > cal3):
        #Si cal2 es mayor que cal3, entonces cal2 es la mayor.
        #If cal2 is greater than cal3, then cal2 is the largest.
        print(f'Es mayor {cal2}')
        #Imprime la mayor.
        #Prints the largest.
        lista1.append(nom)
        #Añade el nombre a la lista temporal.
        #Adds the name to the temporary list.
        lista1.append(cal2)
        #Añade la mayor (cal2).
        #Adds the largest (cal2).
        if(cal1 > cal3):
            #Si cal1 > cal3, entonces cal1 es la del medio y cal3 la menor.
            #If cal1 > cal3, then cal1 is middle and cal3 is smallest.
            print(f'Es el de en medio {cal1}')
            #Imprime la del medio.
            #Prints the middle grade.
            print(f'Es el menor {cal3}')
            #Imprime la menor.
            #Prints the smallest.

lista1.append(cal1)
#Añade cal1 (medio) a la lista temporal.
#Adds cal1 (middle) to the temporary list.
lista1.append(cal3)
#Añade cal3 (menor) a la lista temporal.
#Adds cal3 (smallest) to the temporary list.
lista2.append(lista1)
#Añade la lista temporal a lista2.
#Appends the temporary list to lista2.
```

```
#Si cal3 >= cal1, entonces cal3 es la del medio y cal1 la menor.
#If cal3 >= cal1, then cal3 is middle and cal1 is smallest.
else:
    print(f'Es el de en medio {cal3}')
    #Imprime la del medio.
    #Prints the middle grade.
    print(f'Es el menor {cal1}')
    #Imprime la menor.
    #Prints the smallest.

    lista1.append(cal3)
    #Añade cal3 (medio).
    #Adds cal3 (middle).
    lista1.append(cal1)
    #Añade cal1 (menor).
    #Adds cal1 (smallest).
    lista2.append(lista1)
    #Añade la lista temporal a lista2.
    #Appends the temporary list to lista2.

#Aquí cal3 >= cal2 (cal3 es la mayor).
#Here cal3 >= cal2 (cal3 is the largest).
else:
    print(f'Es el de en medio {cal2}')
    #Imprime la del medio (cal2).
    #Prints the middle grade (cal2).
    print(f'Es el mayor {cal3}')
    #Imprime la mayor (cal3).
    #Prints the largest (cal3).
    print(f'Es el menor {cal1}')
    #Imprime la menor (cal1).
    #Prints the smallest (cal1).
```

```
lista1.append(nom)
#Añade el nombre.
#Adds the name.
lista1.append(cal3)
#Añade la mayor primero.
#Adds the largest first.
lista1.append(cal2)
#Añade la del medio.
#Adds the middle grade.
lista1.append(cal1)
#Añade la menor.
#Adds the smallest.
lista2.append(lista1)
#Guarda la lista ordenada en lista2.
#Stores the ordered list in lista2.
```

```
lista2 = []
if __name__=='__main__':
    while(True):
        inicio()
        a = input('Deseas otra persona s/n: ')
        if a == 'n' or a == 'N':
            print(lista2)
            break
```

PRACTICA 6

4 calificaciones los suma y agrega lista, también promedia en resultado general de los números ingresados

```
'''Ingresas 4 calificaciones, en boton de Promedio se agrega a la lista
la suma del total de las 4 calificaciones, ingresas otras calificaciones
y se agrega a la lista, ademas suma el promedio general de todas
las calificaciones agregadas en la lista'''

from tkinter import *
#Importa todas las funciones de la librería Tkinter (para crear interfaces gráficas).
#Imports all functions from the Tkinter library (for creating graphical interfaces).
from tkinter import messagebox
#Importa el módulo messagebox para mostrar cuadros de mensaje.
#Imports the messagebox module to display message dialogs.

class Principal():
#Define la clase Principal, que controlará toda la ventana y sus funciones.
#Defines the Principal class, which controls the window and its functions.

    def __init__(self):
#Método constructor: se ejecuta al crear el objeto de la clase.
#Constructor method: runs when the class object is created.

        self.ven = Tk()
        #Crea la ventana principal usando Tkinter.
        #Creates the main window using Tkinter.
        self.ven.title('Programa 5 con Ventanas')
        #Establece el título de la ventana.
        #Sets the title of the window.
        self.ven.geometry('600x400')
        #Define el tamaño de la ventana (600x400 píxeles).
        #Sets the size of the window (600x400 pixels).
        self.lista = []
        #Crea una lista vacía para almacenar los promedios individuales.
        #Creates an empty list to store individual averages.
        self.inicio()
        #Llama al método inicio() que crea los elementos visuales de la ventana.
        #Calls the inicio() method that builds the visual components of the window.
```



```
def sumar(self):
#Define una función que suma todos los valores guardados en self.lista.
#Defines a function that sums all values stored in self.lista.
    s = 0
    #Inicializa una variable acumuladora.
    #Initializes an accumulator variable.
    for i in self.lista:
    #Recorre cada elemento de la lista.
    #Loops through each element of the list
        s += i
        #Suma cada número al total acumulado.
        #Adds each number to the accumulated total
    return s
    #Devuelve la suma total.
    #Returns the total sum.
```

```
def promediar(self):
#Función que calcula el promedio de cuatro números y actualiza las etiquetas.
#Function that calculates the average of four numbers and updates the labels.
    try:
    #Intenta ejecutar el siguiente bloque, en caso de error se usa "except".
    #Tries to execute the following block; if error occurs, "except" handles it.
        a = float(self.n1.get())
        #Convierte el texto del primer Entry a número decimal.
        #Converts text from first Entry to a decimal number.
        b = float(self.n2.get())
        #Segundo número.
        #Second number.
        c = float(self.n3.get())
        #Tercer número.
        #Third number.
        d = float(self.n4.get())
        #Cuarto número.
        #Fourth number.
        pro=(a+b+c+d)/4
        #Calcula el promedio de los cuatro números.
        #Calculates the average of the four numbers.
```



```
self.l6.config(text=str(pro))
#Muestra el promedio en la etiqueta 16.
#Displays the average in label 16.
self.lista.append(pro)
#Agrega el promedio calculado a la lista de promedios.
#Adds the calculated average to the list of averages.
self.l7.config(text=str(self.lista))
#Muestra la lista completa de promedios en 17.
#Displays the full list of averages in 17.
self.n1.delete(0,END)
self.n2.delete(0,END)
self.n3.delete(0,END)
self.n4.delete(0,END)
#Limpia las cajas de texto después de calcular el promedio.
#Clears the text boxes after calculating the average.
suma = self.sumar()
#Llama al método que suma todos los promedios almacenados.
#Calls the method that sums all stored averages.
p = suma / len(self.lista)
#Calcula el promedio general de todos los promedios ingresados.
#Calculates the general average of all entered averages.
self.l8.config(text=f'Promedio General: {str(p)}')
#Muestra el promedio general en 18.
#Displays the general average in 18.
```

```
except ValueError:
    messagebox.showerror("Error","Algun dato no es numero")
    #Muestra mensaje de error.
    #Shows error message.
    self.n1.delete(0,END)
    self.n2.delete(0,END)
    self.n3.delete(0,END)
    self.n4.delete(0,END)
    #Limpia las cajas para que el usuario vuelva a ingresar valores.
    #Clears the boxes so the user can re-enter values.
```

```
def salir(self):
    self.ven.destroy()
    #Cierra la ventana.
    #Closes the window.
```

```
def inicio(self):
#Crea todos los elementos visuales (etiquetas, botones, cajas de texto).
#Creates all visual elements (labels, buttons, text boxes).
    l1 = Label(self.ven, text="Escribe un numero: ").place(y=10, x=20)
    #Etiqueta para el primer número.
    #Label for the first number
    l2 = Label(self.ven, text="Escribe un numero: ").place(y=50, x=20)
    #Etiqueta para el segundo número.
    #Label for the second number.
    self.n1 = Entry(self.ven)
    #Caja de texto para el primer número.
    #Text box for the first number.
    self.n1.place(y=10, x=130)
    #Ubica la caja en la ventana.
    #Places the box in the window
    self.n2 = Entry(self.ven)
    #Caja para el segundo número.
    #Box for the second number.
    self.n2.place(y=50, x=130)
    #Coloca la caja en su posición.
    #Positions the box.

    l3 = Label(self.ven, text="Escribe un numero: ").place(y=90, x=20)
    #Tercer número.
    #Third number.
    l4 = Label(self.ven, text="Escribe un numero: ").place(y=130, x=20)
    #Cuarto número.
    #Fourth number.
    self.n3 = Entry(self.ven)
    self.n3.place(y=90, x=130)
    self.n4 = Entry(self.ven)
    self.n4.place(y=130, x=130)

    l5 = Label(self.ven, text="Promedio: ").place(y=150, x=130)
    #Etiqueta para mostrar el promedio individual.
    #Label to display the individual average.
    self.l6 = Label(self.ven, text="0.0")
    #Valor inicial del promedio.
    #Initial value of the average.
    self.l6.place(y=150, x=200)

    b1 = Button(self.ven, text="Promedio", command=self.promediar).place(y=50, x=300)
    #Botón que calcula el promedio individual.
    #Button that calculates the individual average.
    b2 = Button(self.ven, text="Salir", command=self.salir).place(y=90, x=300)
    #Botón que cierra el programa.
    #Button that closes the program
```

```
self.l7 = Label(self.ven, text="[]")
#Muestra la lista de promedios individuales.
#Displays the list of individual averages.
self.l7.place(y=170, x=200)

self.l8 = Label(self.ven, text="Promedio General: 0.0")
#Muestra el promedio general acumulado.
#Displays the accumulated general average.
self.l8.place(y=190, x=130)

self.ven.mainloop()
#Inicia el bucle principal de la interfaz (mantiene la ventana abierta).
#Starts the main GUI loop (keeps the window running).

if __name__ == '__main__':
    app = Principal()
    #Crea una instancia de la clase Principal y lanza la aplicación.
    #Creates an instance of the Principal class and launches the app.
```



Programa 5 con Ventanas

Escribe un numero: Escribe un numero: Escribe un numero: Escribe un numero:

Promedio: 6.5

[7.5, 6.5]

Promedio General: 7.0

PRACTICA 7

Ingresa dos números, se agregan en lista y obtiene el número de mayor y menor

```
'''Ingresas dos numeros, se ingresan a una lista y existen dos botones  
numero mayor y menor lo cual te sale en un mensaje cual es el numero mayor  
y el numero menor GRID'''
```

```
from tkinter import *  
#Importamos las librerías necesarias de tkinter  
#Import the necessary libraries from tkinter  
from tkinter import messagebox  
#Para mostrar mensajes emergentes  
#To show pop-up messages  
  
class Principal():  
#Definimos la clase principal de la aplicación  
#Define the main application class  
    def __init__(self):  
        #Creamos la ventana principal  
        #Create the main window  
        self.ven = Tk()  
        self.ven.title('Programa 6 con Ventanas GRID')  
        #Título de la ventana / Window title  
        self.ven.geometry('450x250')  
        #Tamaño de la ventana / Window size
```

```
#Inicializamos variables  
#Initialize variables  
self.a = 0  
self.b = 0  
self.lista = []  
#Lista donde se guardarán los números  
#List to store numbers  
self.aux1 = 0  
#Auxiliar para encontrar el mayor  
#Helper for max number  
self.aux2 = 0  
#Auxiliar para encontrar el menor / Helper for min number  
self.cont = 0  
#Contador / Counter
```

```
def inicio(self):
#Función que inicia y construye la interfaz gráfica
#Function that builds the GUI
l = Label(self.ven, text="Programa 9")
#Etiqueta del título del programa / Program title label
l.grid(row=1, column=2)
#Colocamos con grid / Place using grid layout

l2 = Label(self.ven, text="Escribe un numero")
l2.grid(row=3, column=1, padx=15, pady=10)
#Etiqueta pidiendo el primer número / Label asking for first number

Label(self.ven, text=" ").grid(row=2, column=2)
#Etiqueta vacía para crear espacio / Empty label for spacing

self.n1 = Entry(self.ven)
self.n1.grid(row=3, column=2)
#Campo de entrada del primer número / Entry field for first number

l3 = Label(self.ven, text="Escribe otro numero")
l3.grid(row=5, column=1, padx=5, pady=5)
#Etiqueta para el segundo número / Label for second number

Label(self.ven, text=" ").grid(row=4, column=2)
#Otra etiqueta vacía para espacio / Another empty label for spacing
```

```
self.n2 = Entry(self.ven)
self.n2.grid(row=5, column=2)
#Campo de entrada del segundo número / Entry field for second number

b1 = Button(self.ven, text="Agregar", command=self.agregar)
b1.grid(row=6, column=1, pady=10)
#Botón para agregar los números a la lista / Button to add numbers to the list

b2 = Button(self.ven, text="Mayor", command=self.mayor)
b2.grid(row=6, column=2)
#Botón para encontrar el mayor / Button to find the largest number

b3 = Button(self.ven, text="Menor", command=self.menor)
b3.grid(row=6, column=3, padx=10)
#Botón para encontrar el menor / Button to find the smallest number

b4 = Button(self.ven, text="Salir", command=self.salir)
b4.grid(row=6, column=4, padx=25)
#Botón para salir del programa / Button to exit the program

self.listaElementos = Label(self.ven, text=" ")
self.listaElementos.grid(row=8, column=2, pady=15)
#Etiqueta donde se mostrará la lista / Label to display the list

self.listview = Listbox(self.ven, height=10,
                        width=15, bg='black',
                        activestyle='dotbox', fg='white')
self.listview.grid(row=2, column=4)
#Listbox para mostrar los elementos de la lista visualmente
#Listbox to show the numbers visually

self.ven.mainloop()
#Ejecuta el bucle principal de la ventana / Run the main window loop
```

```
#Función para obtener el número mayor
#Function to find the largest number
def mayor(self):
    if len(self.lista) > 0:
        #Verifica que la lista no esté vacía / Checks that the list is not empty.
        if self.aux1 < self.lista[self.cont]:
            #Compara el auxiliar aux1 con el elemento en el índice cont.
            #Compares aux1 with the element at index cont
            self.aux1 = self.lista[self.cont]
            #Si el elemento actual es mayor, actualiza aux1.
            #If current element is larger, update aux1.
        self.cont += 1
        #Incrementa el índice para pasar al siguiente elemento.
        #Increments the index to move to the next element.
        if len(self.lista)-1 < self.cont:
            #Si cont ha pasado el último índice (fin del recorrido)...
            #If cont has passed the last index (end of traversal)...
            print(f'El mayor es {self.aux1}')
            #Imprime en consola el mayor encontrado.
            #Prints the found maximum to console.
            messagebox.showinfo('El mayor ', self.aux1)
            #Muestra el mayor en un messagebox (se convertirá a str).
            #Shows the maximum in a messagebox (will be converted to str).
            self.cont = 0
            #Reinicia el contador para futuras búsquedas.
            #Resets the counter for future searches.
    else:
        return self.mayor()
        #Si no se terminó, vuelve a llamarse recursivamente.
        #If not finished, call itself recursively.
    else:
        messagebox.showerror("Error", "La lista esta vacia")
        #Si la lista está vacía / If the list is empty
```



```
def menor(self):
#Función para obtener el número menor
#Function to find the smallest number
    if len(self.lista) > 0:
        #Comprueba que la lista no esté vacía.
        #Checks that the list is not empty.
        for i in self.lista:
            #Recorre cada elemento i dentro de la lista.
            #Iterates through each element i in the list.
            if self.aux2 > i:
                self.aux2 = i
            #Si aux2 es mayor que el elemento actual, actualiza aux2 al valor más pequeño encontrado.
            #Update aux2 to the smaller value found.
        print(f'El menor es {self.aux2}')
        #Imprime el menor en consola.
        #Prints the smallest to console.
        messagebox.showinfo('El mayor ', self.aux2)
        #Muestra un messagebox con el menor
        #Shows a messagebox with the smallest
    else:
        messagebox.showerror("Error","La lista esta vacia")
        #Si vacía, muestra error / If empty, show error.
```

```
def agregar(self):
#Función para agregar números a la lista
#Function to add numbers to the list
    try:
        self.a = int(self.n1.get())
        #Lee el texto de n1, lo convierte a int y lo guarda en self.a.
        #Reads text from n1, converts to int and stores in self.a.
        self.lista.append(self.a)
        #Añade el número a la lista. / Appends the number to the list.
        self.listview.insert(self.listview.size()+1,self.a )
        #Inserta visualmente el número en el Listbox.
        #Inserts the number into the Listbox for display.
        self.n1.delete(0,END)
        #Borra el contenido del campo n1.
        #Clears the n1 entry field.

        self.b = int(self.n2.get())
        #Lee y convierte el valor de n2 a entero.
        #Reads and converts the value of n2 to int.
        self.listview.insert(self.listview.size()+1,self.b )
        #Inserta el segundo número en el Listbox.
        #Inserts the second number into the Listbox.
        self.lista.append(self.b)
        #Añade el segundo número a la lista.
        #Appends the second number to the list.
```



```
self.n2.delete(0,END)
#Borra el campo n2.
#Clears the n2 entry.

print(self.lista)
#Imprime la lista completa en la consola (debug).
#Prints the full list to console (debug).
self.aux2 = self.lista[0]
#Inicializa aux2 con el primer elemento (para la búsqueda del menor).
#Initializes aux2 with the first element (for min search).

self.listaElementos.config(text=f'{self.lista}')
#Actualiza la etiqueta que muestra la lista.
#Updates the label that displays the list.
```

```
except ValueError:
    messagebox.showerror("Error","Algun dato no es numero")
    return self.agregar
#(Inusual) devuelve la función agregar
#(Unusual) returns the agregar function
```

```
def salir(self):
#Método para cerrar la ventana / Method to close the window.
    self.ven.destroy()
#Destruye la ventana y termina la aplicación.
#Destroys the window and ends the application.
```

```
if __name__=='__main__':
    app = Principal()
    #Crea la instancia de la clase Principal.
    #Creates an instance of the Principal class.
    app.inicio()
    #Llama a inicio() para construir la interfaz y arrancar la app.
    #Calls inicio() to build the GUI and start the app.
```

Programa 6 con Ventanas GRID

Programa 9

8
5
9
3

Escribe un numero

Escribe otro numero

[8, 5, 9, 3]

Programa 6 con Ventanas GRID

Programa 9

8
5
9
3

Escribe un numero

Escribe otro numero

[8, 5, 9, 3]

El mayor

9

Programa 6 con Ventanas GRID

Programa 9

8
5
9
3

Escribe un numero

Escribe otro numero

[8, 5, 9, 3]

El mayor

3

PRACTICA 8

Ingresa dos números random y obtiene el número mayor y menor

```
'''Ingresa dos numeros random, se ingresan a una lista y existen dos botones  
numero mayor y menor lo cual te sale en un mensaje cual es el numero mayor  
y el numero menor PLACE'''
```

```
from tkinter import *  
from tkinter import messagebox  
import random  
#Importa el módulo random para generar números aleatorios  
#Imports random library to generate random numbers
```

```
class Principal():  
#Define una clase llamada Principal / Defines a class called Principal  
    def __init__(self):  
        #Método constructor que se ejecuta al crear el objeto  
        #Constructor method that runs when the object is created  
        self.ven = Tk()  
        #Crea la ventana principal / Creates the main window  
        self.ven.title('Programa 7 con Ventanas PLACE')  
        self.ven.geometry('550x250')  
  
        self.a = 0  
        #Variable para el primer número / Variable for the first number  
        self.b = 0  
        #Variable para el segundo número / Variable for the second number  
        self.lista = []  
        #Lista para guardar los números / List to store numbers  
        self.aux1 = 0  
        #Variable auxiliar para encontrar el número mayor  
        #Auxiliary variable to find the largest number  
        self.aux2 = 0  
        #Variable auxiliar para encontrar el número menor  
        #Auxiliary variable to find the smallest number  
        self.cont = 0  
        #Contador usado en la función mayor / Counter used in the "mayor" (max) function
```

```
def inicio(self):
#Método que construye la interfaz / Method that builds the interface
    l = Label(self.ven, text="Programa 10")
    l.place(x=150, y=20)
    #Crea una etiqueta / Creates a label

    l2 = Label(self.ven, text="Escribe un numero")
    l2.place(x=100, y=50)
    #Crea otra etiqueta / Creates another label

    self.n1 = Entry(self.ven)
    self.n1.place(x=100, y=75)
    #Crea una caja de texto / Creates an input box

    l3 = Label(self.ven, text="Escribe otro numero")
    l3.place(x=250, y=50)
    #Crea una tercera etiqueta / Creates a third label

    self.n2 = Entry(self.ven)
    self.n2.place(x=250, y=75)
    #Crea otra caja de texto / Creates another input box

    b1 = Button(self.ven, text="Agregar", command=self.agregar)
    b1.place(x=110, y=110)
    #Coloca el botón / Places the button
```

```
b2 = Button(self.ven, text="Mayor", command=self.mayor)
b2.place(x=180, y=110)
#Botón para mostrar el número mayor / Button to show the largest number

b3 = Button(self.ven, text="Menor", command=self.menor)
b3.place(x=250, y=110)
# Botón para mostrar el número menor / Button to show the smallest number
(parameter) self: Self@Principal

b4 = Button(self.ven, text="Salir", command=self.salir)
b4.place(x=310, y=110, width=50)
#Botón para salir / Exit button

self.listaElementos = Label(self.ven, text=" ")
self.listaElementos.place(x=150, y=180)
#Etiqueta para mostrar los elementos de la lista / Label to show the list elements
self.listview = Listbox(self.ven, height=10,
                        width=15, bg='black',
                        activestyle='dotbox', fg='white')
#Caja de lista para mostrar los números agregados / Listbox to display added numbers
self.listview.place(x=410, y=20)
#Coloca la lista / Places the listbox

self.ven.mainloop()
#Mantiene la ventana abierta / Keeps the window open

def mayor(self):
#Función para encontrar el número mayor / Function to find the largest number
    if len(self.lista) > 0:
        #Verifica si la lista tiene elementos / Checks if the list has elements
        if self.aux1 < self.lista[self.cont]:
            #Si el número actual es mayor que aux1 / If the current number is greater than aux1
            self.aux1 = self.lista[self.cont]
            #Actualiza el número mayor / Updates the largest number
        self.cont += 1
        #Aumenta el contador / Increases the counter
        if len(self.lista)-1 < self.cont:
            #Si ya recorrió toda la lista / If the entire list has been checked
            print(f'El mayor es {self.aux1}')
            #Muestra en consola / Prints to console
            messagebox.showinfo('El mayor ', self.aux1)
            #Muestra en ventana emergente / Shows in a popup window
            self.cont = 0
            #Reinicia el contador / Resets the counter
        else:
            return self.mayor()
            #Llama la función de nuevo (recursividad) / Calls the function again (recursion)

    else:
        messagebox.showerror("Error", "La lista esta vacia")
        #Error si no hay datos / Error if the list is empty
```

```
def menor(self):
#Función para encontrar el número menor / Function to find the smallest number
    if len(self.lista) > 0:
        #Verifica si hay elementos / Checks if there are elements
        for i in self.lista:
            #Recorre la lista / Iterates through the list
            if self.aux2 > i:
                #Si el valor actual es menor / If the current value is smaller
                self.aux2 = i
            #Actualiza el número menor / Updates the smallest number
        print(f'El menor es {self.aux2}')
        #Muestra el resultado / Prints the result
        messagebox.showinfo('El mayor ', self.aux2)
        #Muestra ventana con el resultado / Shows popup with the result
    else:
        messagebox.showerror("Error", "La lista esta vacia")
        #Muestra error si no hay datos / Shows error if list is empty
```

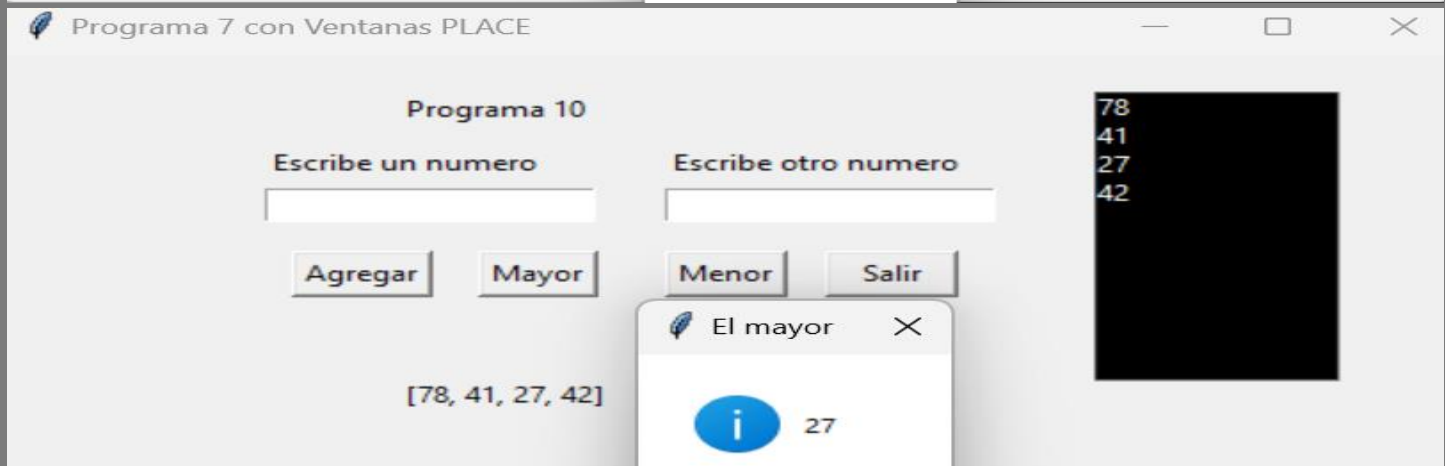
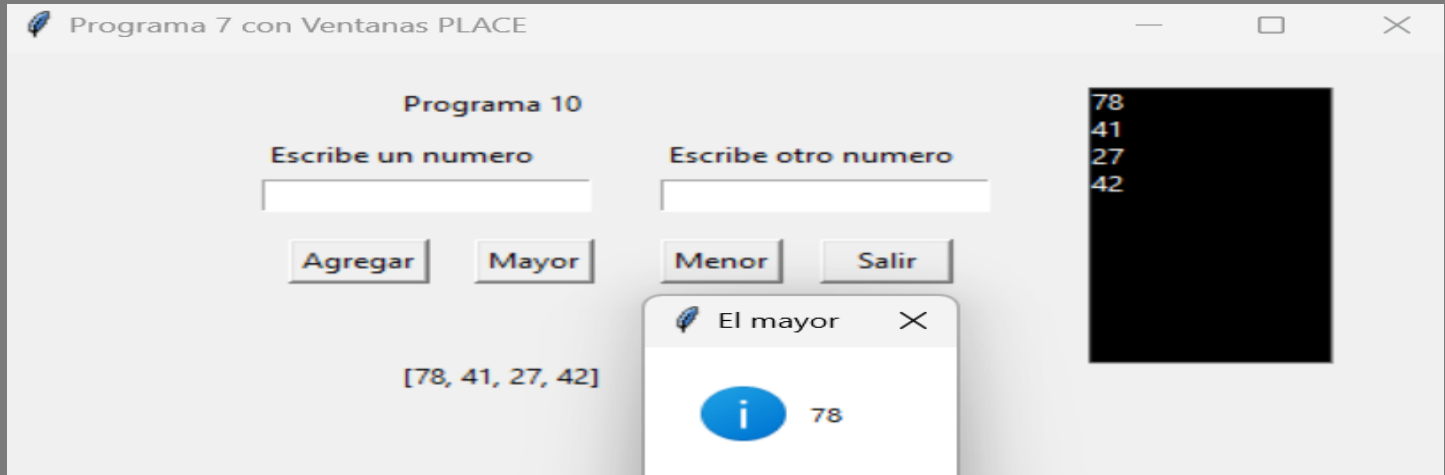
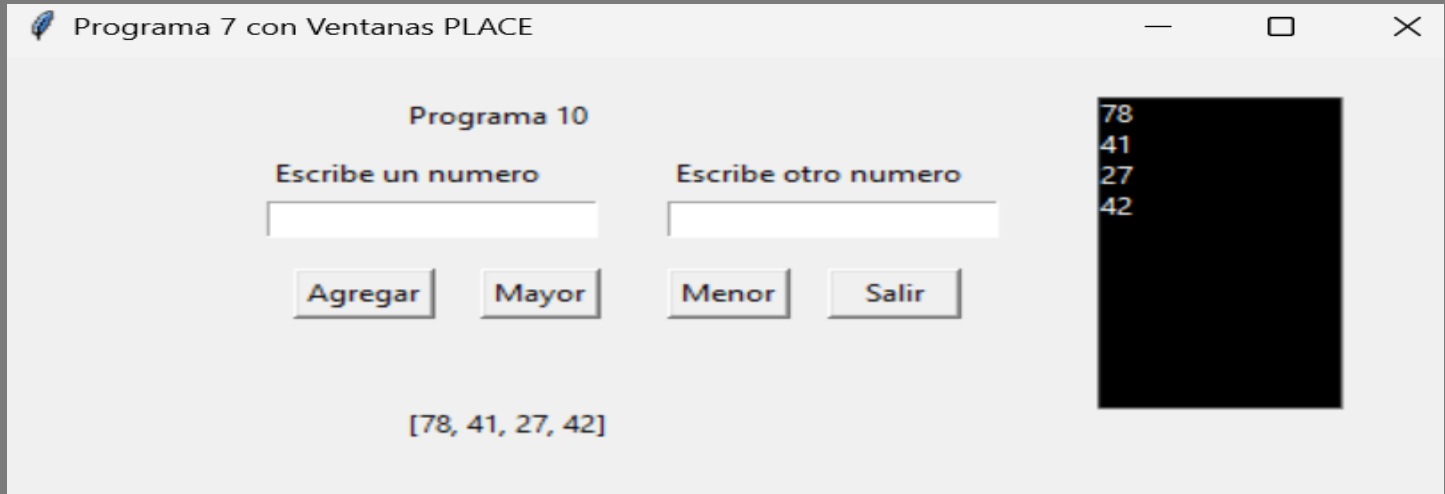
```
def agregar(self):
#Función para agregar números aleatorios / Function to add random numbers
    try:
        valor = random.randint(1,100)
        #Genera un número aleatorio / Generates a random number
        self.lista.append(valor)
        #Lo agrega a la lista / Adds it to the list
        self.listview.insert(self.listview.size()+1,valor )
        #Lo muestra en el Listbox / Displays it in the listbox

        print(self.lista)
        #Muestra la lista en consola / Prints the list in console
        self.aux2 = self.lista[0]
        #Inicializa el menor con el primer número / Initializes smallest with first number

        self.listaElementos.config(text=f'{self.lista}')
        #Actualiza el Label / Updates the label

    except ValueError:
        messagebox.showerror("Error", "Algun dato no es numero")
        #Error si ocurre un problema / Shows error if something goes wrong
        return self.agregar
        #Retorna la función / Returns the function
```

```
def salir(self):  
    #Función para cerrar la ventana / Function to close the window  
    self.ven.destroy()  
    # Destruye la ventana principal / Destroys the main window  
  
if __name__=='__main__':  
    app = Principal()  
    #Crea un objeto de la clase Principal / Creates an object of the Principal class  
    app.inicio()  
    #Llama al método inicio() para mostrar la ventana / Calls inicio() to display the window
```



PRACTICA 9

Validar números, letras y signos

```
'''REPASO'''
from tkinter import *
from tkinter import messagebox
from validarp9 import Validar
#Importa la clase Validar desde otro archivo / Imports the Validar class from another file

class Principal():
    def __init__(self):
        self.ventana = Tk()
        self.ventana.geometry("400x200")
        self.lista = []
        self.valid = Validar()

    def inicio(self):
        Label(self.ventana, text="Programa de python con TKInter").place(x=100, y=20)
        Label(self.ventana, text="Escribe un dato").place(x=50, y=50)
        self.dato = Entry(self.ventana)
        self.dato.place(x=150, y=50, height=20)
        Button(self.ventana, text="Validar", command=self.ValidarDatos).place(x=150, y=90, width=120)
        self.mostrar = Label(self.ventana, text=" ")
        self.mostrar.place(x=20, y=150)

        self.ventana.mainloop()

    def ValidarDatos(self):
        val = self.dato.get()
        #Obtiene el dato de la caja de texto / Gets the value from the entry widget
        if val != "":
            #Verifica que no esté vacía / Checks that it is not empty
            self.Revisar(val)
            #Llama a Revisar para imprimir el valor / Calls Revisar to print the value
            self.lista.append(val)
            #Agrega el valor a la lista / Appends the value to the list
            self.dato.delete(0,END)
            #Limpia la caja de texto / Clears the entry widget
            self.mostrar.config(text=f'{self.lista}')
            #Actualiza el label con la lista / Updates the label with the list
```



```
'''TIPOS DE VALIDACION'''
#respuesta = self.valid.ValidarAscii(val)
respuesta = self.valid.ValidarConError(val)
#respuesta = self.valid.ValidarConString(val)

messagebox.showinfo("Validar datos",f'El datos es: {respuesta}')
#Muestra el resultado de la validación / Shows the validation result
else:
    messagebox.showerror("Error","Caja de texto esta vacia")

def Revisar(self, v):
    #Función auxiliar para imprimir en consola / Helper function to print to console
    print(v)
    #Imprime el dato / Prints the value
```

```
if __name__=='__main__':
    app = Principal()
    app.inicio()
```

```
class Validar():
    def __init__(self):
        self.index = 0

    '''LETRAS MAYUSCULAS Y NUMEROS'''
    def ValidarAscii(self, valor):
        con = 0
        con2 = 0
        for i in valor:
            if ord(i) >= 48 and ord(i) <=57:
                con += 1
            if (ord(i) >=65 and ord(i) <=90) or(ord(i) >=197 and ord(i) <=122):
                con2 += 1
        if con == len(valor):
            return "Numeros"
        elif con2 == len(valor):
            return "Letras"
        else:
            return "Letras y Numeros"
```

```
'''NUMEROS DECIMALES Y ENTEROS'''
```

```
def ValidarConError(self, valor):  
    a = 0  
    b = 0.0  
    try:  
        a= int(valor)  
        return "numeros"  
    except ValueError:  
        try:  
            b = float(valor)  
            return "Es numero real o decimales"  
        except ValueError:  
            return "letras o numeros"
```

```
'''VALIDAR CORREO'''
```

```
def ValidarConString(self, valor):  
    print(valor)  
    if self.index < len(valor):  
        if valor[self.index] == "@":  
            return "Si es un correo"  
        else:  
            if self.index < len(valor):  
                self.index += 1  
                return self.ValidarConString(valor)  
            else:  
                return "No es un correo"  
    else:  
        return "No es un correo"
```



```
def inicio(self):

    self.min1 = Entry(self.ven)
    self.min1.place(x=80, y=75)

    self.may2 = Entry(self.ven)
    self.may2.place(x=210, y=75)

    self.num3 = Entry(self.ven)
    self.num3.place(x=340, y=75)

    b1 = Button(self.ven, text="Validar", command=self.ValidarTodo)
    b1.place(x=110, y=110)

    b2 = Button(self.ven, text="Agregar", command=self.agregar)
    b2.place(x=180, y=110)

    self.listview = Listbox(self.ven, height=10,
                             width=15, bg='black',
                             activestyle='dotbox', fg='white')
    self.listview.place(x=510, y=20)

    Label(self.ven, text="Elementos en la lista:").place(x=450, y=210)

    self.contador = Label(self.ven, text="0")
    self.contador.place(x=580, y=210)

    self.ven.mainloop()
```

```
def agregar(self):
    if self.valido1 and self.valido2 and self.valido3:
        val1 = self.min1.get().strip()
        val2 = self.may2.get().strip()
        val3 = self.num3.get().strip()

        if val1 == "" or val2 == "" or val3 == "":
            messagebox.showerror("Error", "Debes llenar las tres cajas antes de agregar.")
            return

        if self.valido1 and self.valido2 and self.valido3:
            concatenado = f"{val1} {val2} {val3}"
            self.lista.append(concatenado)
            self.listview.insert(END, concatenado)

            self.contador.config(text=str(len(self.lista)))

            self.min1.delete(0, END)
            self.may2.delete(0, END)
            self.num3.delete(0, END)

            self.valido1 = self.valido2 = self.valido3 = False
            messagebox.showinfo("Agregado", "Elemento agregado correctamente")
        else:
            messagebox.showerror("Error", "Primero valida correctamente los tres campos.")

def ValidarTodo(self):
    val1 = self.min1.get().strip()
    val2 = self.may2.get().strip()
    val3 = self.num3.get().strip()

    if val1 == "" and val2 == "" and val3 == "":
        messagebox.showerror("Error", "Las tres cajas están vacías.")
        return

    self.valido1 = self.valido2 = self.valido3 = False

    if val1 != "":
        if self.valid.ValidarMinusculas(val1):
            self.valido1 = True
        else:
            messagebox.showerror("Error", "Solo letras minúsculas permitidas.")
            self.min1.delete(0, END)

    if val2 != "":
        if self.valid.ValidarMayusculas(val2):
            self.valido2 = True
        else:
            messagebox.showerror("Error", "Solo letras MAYÚSCULAS permitidas.")
            self.may2.delete(0, END)

    if val3 != "":
        if self.valid.ValidarNumeros(val3):
            self.valido3 = True
```

```
if val3 != "":
    if self.valid.ValidarNumeros(val3):
        self.valido3 = True
    else:
        messagebox.showerror("Error", "Solo números permitidos.")
        self.num3.delete(0, END)

if val1 == "" or val2 == "" or val3 == "":
    messagebox.showerror("Error", "Faltan campos por llenar.")
    return

if self.valido1 and self.valido2 and self.valido3:
    messagebox.showinfo("Validación correcta", "Todas las entradas son válidas")
```

```
if __name__ == '__main__':
    app = Principal()
    app.inicio()
```

Examen Repaso

Elementos en la lista: 0

REPASO 2

```
2  from tkinter import *
3  from tkinter import messagebox
4  from validarrepaso2 import Validar1
5
6  contador = 0
7  class Principal():
8      def __init__(self):
9          self.ven = Tk()
10         self.ven.title('Examen Repaso2')
11         self.ven.geometry('500x400')
12         self.valid = Validar1()
13         self.valido1 = self.valido2 = self.valido3 = False
14         self.lista = []
15
16
17     def inicio(self):
18         Label(self.ven, text="REGISTRO DE ESTUDIANTES").place(x=110, y=10)
19
20         Label(self.ven, text="Nombre:").place(x=60, y=50)
21         Label(self.ven, text="Edad:").place(x=60, y=80)
22         Label(self.ven, text="Correo:").place(x=60, y=110)
23
24         self.n1 = Entry(self.ven)
25         self.n1.place(x=120, y=50)
26         self.n2 = Entry(self.ven)
27         self.n2.place(x=120, y=80)
28         self.n3 = Entry(self.ven)
29         self.n3.place(x=120, y=110)
30
31 b1 = Button(self.ven, text="Validar", command=self.validar)
32 b1.place(x=300, y=50)
33 b2 = Button(self.ven, text="Agregar", command=self.agrega)
34 b2.place(x=300, y=80)
35 b3 = Button(self.ven, text="Eliminar", command=self.eliminar)
36 b3.place(x=360, y=80)
37 b4 = Button(self.ven, text="Eliminar todo", command=self.eliminartodo)
38 b4.place(x=360, y=50)
39 b5 = Button(self.ven, text="Ordenar por Nombre", command=lambda: self.ordenar("nombre"))
40 b5.place(x=300, y=150)
41 b6 = Button(self.ven, text="Ordenar por Edad", command=lambda: self.ordenar("edad"))
42 b6.place(x=300, y=180)
43 b7 = Button(self.ven, text="Ordenar por Correo", command=lambda: self.ordenar("correo"))
44 b7.place(x=300, y=210)
45
46 self.listview = Listbox(self.ven, height=5,
47                          width=30, bg='black',
48                          activestyle='dotbox', fg='white')
49 self.listview.place(x=80, y=150)
50
51 Label(self.ven, text="Elementos en la lista").place(x=300, y=110)
```

```
self.contador = Label(self.ven, text="0")
self.contador.place(x=355, y=130)
self.mayores = Label(self.ven, text="Mayores de edad: 0")
self.mayores.place(x=100, y=250)
self.menores = Label(self.ven, text="Menores de edad: 0")
self.menores.place(x=100, y=280)
```

```
self.ven.mainloop()
```

```
def estadisticas(self):
```

```
    mayores = sum(1 for item in self.lista if int(item.split(" , ")[1]) >= 18)
    menores = sum(1 for item in self.lista if int(item.split(" , ")[1]) < 18)
```

```
    self.contador.config(text=str(len(self.lista)))
    self.mayores.config(text=f"Mayores de edad: {mayores}")
    self.menores.config(text=f"Menores de edad: {menores}")
```

```
def ordenar(self, criterio):
```

```
    if not self.lista:
        messagebox.showwarning("Aviso", "No hay elementos para ordenar.")
        return
```

```
    registros_split = [item.split(" , ") for item in self.lista]
```

```
    if criterio == "nombre":
        registros_split.sort(key=lambda x: x[0].lower())
    elif criterio == "edad":
        registros_split.sort(key=lambda x: int(x[1]))
    elif criterio == "correo":
        registros_split.sort(key=lambda x: x[2].lower())
```

```
    self.lista = [" , ".join(r) for r in registros_split]
    self.listview.delete(0, END)
    for i, item in enumerate(self.lista, start=1):
        self.listview.insert(END, f"{i}. {item}")
```

```
    self.estadisticas()
```

```
def eliminartodo(self):
```

```
    if not self.lista:
        messagebox.showwarning("Aviso", "No hay estudiantes en la lista para eliminar.")
        return
```

```
    confirmar = messagebox.askyesno("Confirmar", "¿Deseas eliminar TODOS los estudiantes?")
    if not confirmar:
        return
```



```
self.lista.clear()
self.listview.delete(0, END)
self.mayores.config(text="Mayores de edad: 0")
self.menores.config(text="Menores de edad: 0")

self.contador.config(text="0")

messagebox.showinfo("Eliminado", "Todos los estudiantes fueron eliminados correctamente.")
```

```
def eliminar(self):
    seleccion = self.listview.curselection()
    if not seleccion:
        messagebox.showerror("Error", "Debes seleccionar un estudiante para eliminar.")
        return

    indice = seleccion[0]
    estudiante = self.listview.get(indice)

    confirmar = messagebox.askyesno("Confirmar", f"¿Deseas eliminar:\n{estudiante}?")
    if not confirmar:
        return
```

```
self.listview.delete(indice)
if indice < len(self.lista):
    self.lista.pop(indice)

self.listview.delete(0, END)
for i, item in enumerate(self.lista, start=1):
    self.listview.insert(END, f"{i}. {item}")

self.estadisticas()
self.contador.config(text=str(len(self.lista)))

messagebox.showinfo("Eliminado", f"El estudiante fue eliminado correctamente.")
```

```
def agrega(self):
    val1 = self.n1.get().strip()
    val2 = self.n2.get().strip()
    val3 = self.n3.get().strip()

    if val1 == "" or val2 == "" or val3 == "":
        messagebox.showerror("Error", "Debes llenar las tres cajas antes de agregar.")
        return

    if self.valido1 and self.valido2 and self.valido3:
        concatenado = f"{val1} , {val2} , {val3}"
```

```
if concatenado in self.lista:
    messagebox.showerror("Duplicado", "Este estudiante ya está registrado.")
    return

numero = len(self.lista) + 1
registro_numerado = f"{numero}. {concatenado}"

self.lista.append(concatenado)
self.listview.insert(END, registro_numerado)

self.contador.config(text=str(len(self.lista)))
self.estadisticas()

self.n1.delete(0, END)
self.n2.delete(0, END)
self.n3.delete(0, END)

self.valido1 = self.valido2 = self.valido3 = False
messagebox.showinfo("Agregado", "Elemento agregado correctamente")
else:
    messagebox.showerror("Error", "Primero valida correctamente los tres campos.")
```

```
def validart(self):
    val1 = self.n1.get().strip()
    val2 = self.n2.get().strip()
    val3 = self.n3.get().strip()

    if val1 == "" and val2 == "" and val3 == "":
        messagebox.showerror("Error", "Las tres cajas están vacías.")
        return

    self.valido1 = self.valido2 = self.valido3 = False

    if val1 != "":
        if self.valid.ValidarLetras(val1):
            self.valido1 = True
            messagebox.showinfo("Correcto", "Son letras")
        else:
            messagebox.showerror("Error", "Solo letras.")
            self.n1.delete(0, END)

    if val2 != "":
        if self.valid.ValidarNumeros(val2):
            self.valido2 = True
            messagebox.showinfo("Correcto", "Son Numeros")
        else:
            messagebox.showerror("Error", "Solo Numeros.")
            self.n2.delete(0, END)
```

```
if val3 != "":
    if self.valid.ValidarCorreo(val3):
        self.valido3 = True
        messagebox.showinfo("Correcto", "Contiene el arroba (@)")
    else:
        messagebox.showerror("Error", "No contiene el arroba (@).")
        self.n3.delete(0, END)

if val1 == "" or val2 == "" or val3 == "":
    messagebox.showerror("Error", "Faltan campos por llenar.")
    return

if self.valido1 and self.valido2 and self.valido3:
    messagebox.showinfo("Validación correcta", "Todas las entradas son válidas")

if __name__ == '__main__':
    app = Principal()
    app.inicio()
```



Examen Repaso2



REGISTRO DE ESTUDIANTES

Nombre: Edad: Correo: 

Validar

Eliminar todo

Agregar

Eliminar

Elementos en la lista

0

Ordenar por Nombre

Ordenar por Edad

Ordenar por Correo

Mayores de edad: 0

Menores de edad: 0

REPASO 3

```
from tkinter import *
from tkinter import messagebox

class Principal():
    def __init__(self):
        self.ven = Tk()
        self.ven.title('Examen Repaso3')
        self.ven.geometry('400x300')

    def inicio(self):
        Label(self.ven, text="Palabra:").place(x=80, y=50)

        self.n1 = Entry(self.ven)
        self.n1.place(x=140, y=50)

        b1 = Button(self.ven, text="Palindromo", command=self.pali)
        b1.place(x=300, y=50)

        self.ven.mainloop()

    def pali(self):
        palabra = self.n1.get().strip().lower()

        palabra2= "".join(c for c in palabra if c.isalpha())
        if palabra2 == palabra2[::-1]:
            messagebox.showinfo("Resultado",f"'{palabra}' si es palindromo")
        else:
            messagebox.showerror("Resultado",f"'{palabra}'no es palindromo")

if __name__=='__main__':
    app = Principal()
    app.inicio()
```

Palabra:

REPASO 4

La palabra debe contener todas las vocales

```
from tkinter import *
from tkinter import messagebox

class Principal():
    def __init__(self):
        self.ven = Tk()
        self.ven.title('Examen Repaso3')
        self.ven.geometry('400x300')

    def inicio(self):
        Label(self.ven, text="Palabra:").place(x=80, y=50)

        self.n1 = Entry(self.ven)
        self.n1.place(x=140, y=50)

        b1 = Button(self.ven, text="Vocales", command=self.voca)
        b1.place(x=300, y=50)

        self.ven.mainloop()

    def texto(self, texto):
        return "".join(c for c in texto if c.isalpha())

    def voca(self):
        frase = self.n1.get().strip()
        if not frase:
            messagebox.showerror("Error", "Debes ingresar una frase")
            return

        frase_limpia = self.texto(frase)
        vocales = "aeiouAEIOU" # Incluye mayúsculas y minúsculas
        faltantes = [v for v in "aeiou" if v not in frase_limpia.lower()] # solo para informar cuáles faltan

        if not faltantes:
            messagebox.showinfo("Vocales", f"'{frase}' contiene todas las vocales")
        else:
            messagebox.showerror("Vocales", f"'{frase}' no contiene todas las vocales. Faltan: {'', '.join(faltantes)}")
```

```
if __name__ == '__main__':  
    app = Principal()  
    app.inicio()
```

Palabra:

Vocales

PARCIAL 3

PRACTICA 1

RadioButton con ordenar método Pilas y colas, Eliminar método Burbuja o selección

```
'''Validar Numeros, solo ingresar 2 digitos, eliminar un dato que selecciones y  
ordenar los numeros en pilas o colas y eliminar en burbuja o seleccion, depense de su  
seleccion con el radiobutton'''
```

```
from tkinter import *  
from tkinter import messagebox  
from Validaciones1 import Validar  
import numpy as np
```

```
'''VENTANA'''
```

```
class Principal():  
    def __init__(self):  
        self.val = Validar()  
        self.ven = Tk()  
        ancho = 320  
        alto = 210  
        ventana_alto = self.ven.winfo_screenwidth()  
        ventana_ancho = self.ven.winfo_screenheight()  
        x = (ventana_alto // 2) - (ancho // 2)  
        y = (ventana_ancho // 2) - (alto // 2)  
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")  
        self.lis = []  
        #Lista para almacenar datos  
        #List to store input numbers
```

```
def validarCaja(self):  
    valor = self.dato.get()  
    #Obtiene el valor escrito en la caja de texto  
    #Gets the entered value from the Entry widget  
    if (self.val.ValidarNumeros(valor)):  
        #Verifica si son números  
        #Checks if input is numeric  
        if (self.val.ValidarEntradas(valor)):  
            #Verifica si solo tiene dos dígitos  
            #Checks if input has only two digits  
            self.lista.insert(self.lista.size()+1, valor)  
            #Inserta el valor en la lista  
            #Inserts value into the Listbox  
            self.dato.delete(0,END)  
            #Limpia la caja de texto  
            #Clears the Entry box  
        else:  
            messagebox.showerror("Error","Solo se permite dos digitos")  
            self.dato.delete(0,END)  
            #Error si no son dos dígitos  
            #Error if value isn't two digits  
    else:  
        messagebox.showerror("Error","No son numeros")  
        self.dato.delete(0,END)  
        #Error si no es número  
        #Error if input is not numeric
```



```
self.label.config(text=f'Elementos en la lista: {str(self.lista.size())}')
#Actualiza etiqueta con cantidad de elementos
#Updates label with number of elements

def eliminarDato(self):
    if self.lista.size() <= 0:
        messagebox.showerror("Error","La lista esta vacia")
        return
    #Si la lista está vacía muestra error
    #Error if the list is empty
    if self.modos.get() == 'Pilas':
        #Si está en modo PILAS (LIFO)
        #If using STACK mode (LIFO)
        self.lista.delete(self.lista.size()-1)
        #Último entra, primero sale
        #Last in, first out
    else:
        # Modo COLAS (FIFO)
        # QUEUE mode (FIFO)
        self.lista.delete(0)
    self.label.config(text=f'Elementos en la lista: {str(self.lista.size())}')
    #Primero entra, primero sale
    #First in, first out
```

```
def ordenar(self):
    self.lis = list(self.lista.get(0,END))
    #Obtiene todos los elementos del Listbox
    #Gets all items from Listbox
    if len(self.lis) <= 0:
        messagebox.showerror("Error","Lista vacia")
    #Si la lista está vacía da error
    #Error if list is empty
    metodo = self.modoorden.get()
    #Obtiene el método de ordenamiento seleccionado
    #Gets selected sorting method

    '''BURBUJA'''
    if metodo == "Burbuja":
        #Recorridos para ordenar/Sorting loops
        for i in range(0,len(self.lis)):
            for x in range(i,len(self.lis)-1):
                #Compara elementos adyacentes
                #Compares adjacent elements
                if self.lis[x] > self.lis[x+1]:
                    aux = self.lis[x]
                    self.lis[x] = self.lis[x+1]
                    self.lis[x+1] = aux
            print(self.lis)
        self.lista.delete(0,END)
        for i in self.lis:
            self.lista.insert(self.lista.size()+1,i)
```

```

'''SELECCION'''
else:
    p = 0
    #Posición del mayor
    #Position of largest element
    for i in range(0,len(self.lis)):
        #Posición del mayor
        #Position of largest element
        aux = int(self.lis[i])
        p = i
        for x in range(i,len(self.lis)):
            # print(self.lis[x])
            if aux < int(self.lis[x]):
                aux = int(self.lis[x])
                p = x
        #Busca el número mayor
        #Searches for the largest number
        self.lis[p] = self.lis[i]
        self.lis[i] = str(aux)
        # Intercambia posiciones/ Swaps positions
    print(self.lis)
    self.lista.delete(0,END)
    for i in self.lis:
        self.lista.insert(self.lista.size()+1,i)
    #Actualiza la lista visual
    #Updates the visual list

```

```

'''INTERFAZ'''
def inicio(self):
    self.dato = Entry(self.ven)
    self.dato.place(x=50, y=10)

    self.modoo = StringVar(value="Pilas")
    Radiobutton(self.ven, text="Pilas", variable=self.modoo, value="Pilas").place(x=50, y=40)
    Radiobutton(self.ven, text="Colas", variable=self.modoo, value="Colas").place(x=100, y=40)
    self.modooorden = StringVar(value="Burbuja")
    Radiobutton(self.ven, text="Burbuja", variable=self.modooorden, value="Burbuja").place(x=30, y=60)
    Radiobutton(self.ven, text="Seleccion", variable=self.modooorden, value="Seleccion").place(x=100, y=60)

    Button(self.ven, text="Validar", command=self.validarCaja, width=10).place(x=100, y=100)
    Button(self.ven, text="Eliminar", command=self.eliminarDato, width=10).place(x=100, y=130)
    Button(self.ven, text="Ordenar", command=self.ordenar, width=10).place(x=100, y=160)

    self.label = Label(text="Numero")
    self.label.place(x=5, y=80)
    self.lista= Listbox(self.ven, height=10, width=10, bg="white", font=("Helvetica",12))
    self.lista.place(x=190, y=10)

    self.ven.mainloop()

```

```

if __name__=='__main__':
    app = Principal()
    app.inicio()

```

tk

☒ Pilas ☐ Colas

☒ Burbuja ☐ Seleccion

Numero

Validar

Eliminar

Ordenar

```
class Validar():
    def __init__(self):
        self.con = 0

    def ValidarNumeros(self, num):
        if self.con >= len(num):
            self.con = 0
            return True
        if ord(num[self.con])>=47 and ord(num[self.con]) <= 58:
            self.con += 1
            return self.ValidarNumeros(num)
        else:
            self.con = 0
            return False

    def ValidarLetra(self, dato):
        if ord(dato[0]) >= 65 and (dato[0]) <= 90):
            return True
        else:
            return False

    def ValidarEntradas(self, dato):
        if dato == "":
            return False
        if len(dato) == 2:
            return True
        else:
            return False
```

PRACTICA 2

Realizar RFC Validando letras y números, agregándolos en la lista y ordenar en método pila o cola

```
'''hacer un programa que lea nombre, apellido paterno y materno
en 3 cajas sepoaradas,ademas leer dia, mes y año en 3 cajas de texto
separadas.
al presionar un boton se agregara a un lisbox el rfc de la persona, ademas
tendra dos botones para eleminir elementos de listbox mediante pilas o colas'''

'''IMPORTACIONES'''
from tkinter import *
from tkinter import messagebox
from validacionesp2 import Validar

'''VENTANA, TAMAÑO'''
class Principal():
    def __init__(self):
        self.val = Validar()
        self.ven = Tk()
        ancho = 420
        alto = 310
        ventana_alto = self.ven.winfo_screenwidth()
        ventana_ancho = self.ven.winfo_screenheight()
        x = (ventana_alto // 2) - (ancho //2)
        y = (ventana_ancho // 2) - (alto //2)
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")
        self.datos = []
```

```
'''DISEÑO DE VENTANA'''
```

```
def inicio(self):
```

```
    self.label = Label(text="Nombre")
    self.label.place(x=5, y=10)
    self.nombre = Entry(self.ven)
    self.nombre.place(x=60, y=10)
```

```
    self.label = Label(text="Apellido Paterno")
    self.label.place(x=5, y=35)
    self.Pat = Entry(self.ven)
    self.Pat.place(x=110, y=35)
```

```
    self.label = Label(text="Apellido Materno")
    self.label.place(x=5, y=65)
    self.Mat = Entry(self.ven)
    self.Mat.place(x=110, y=65)
```

```
    self.label = Label(text="Dia de Nacimiento")
    self.label.place(x=5, y=95)
    self.Dia = Entry(self.ven)
    self.Dia.place(x=110, y=95)
```

```
    self.label = Label(text="Mes de Nacimiento")
    self.label.place(x=5, y=125)
    self.Mes = Entry(self.ven)
    self.Mes.place(x=115, y=125)
```

```
    self.label = Label(text="Año de Nacimiento")
    self.label.place(x=5, y=155)
    self.An = Entry(self.ven)
    self.An.place(x=115, y=155)
```

```
    Button(self.ven, text="Agregar", command=self.agregar, width=10).place(x=120, y=210)
    Button(self.ven, text="Eliminar", command=self.eliminard, width=10).place(x=30, y=210)
    Button(self.ven, text="Ordenar", command=self.ordenar, width=10).place(x=210, y=210)
```

```
    self.lista= Listbox(self.ven, height=10, width=10, bg="white", font=("Helvetica",12))
    self.lista.place(x=270, y=10)
```

```
    self.modos = StringVar(value="Pilas")
    Radiobutton(self.ven, text="Pilas", variable=self.modos, value="Pilas").place(x=60, y=250)
    Radiobutton(self.ven, text="Colas", variable=self.modos, value="Colas").place(x=120, y=250)
```

```
    self.ven.mainloop()
```

```
def agregar(self):
    nombre = self.nombre.get().strip().upper()
    pat = self.Pat.get().strip().upper()
    mat = self.Mat.get().strip().upper()
    #Obtiene el nombre y apellidos, elimina espacios y convierte a mayúsculas
    #Gets name and surnames, strips spaces and converts to uppercase
    dia = self.Dia.get().strip()
    mes = self.Mes.get().strip()
    anio = self.An.get().strip()
    #Obtiene la fecha (día, mes, año)
    #Gets date values (day, month, year)
    if not (nombre and pat and mat and dia and mes and anio):
        messagebox.showerror("Error", "Todos los campos son obligatorios.")
        return
    #Verifica si algún campo está vacío
    #Checks if any field is empty
    if not (self.val.ValidarLetra(nombre) and self.val.ValidarLetra(pat) and self.val.ValidarLetra(mat)):
        messagebox.showerror("Error", "Los nombres y apellidos deben contener solo letras")
        return
    #Verifica que nombre y apellidos sean solo letras
    #Verifies that names contain letters only
    if not (self.val.ValidarNumeros(dia) and self.val.ValidarNumeros(mes) and self.val.ValidarNumeros(anio)):
        messagebox.showerror("Error", "No son numeros")
        return
    #Verifica que la fecha sean números
    #Checks that date fields are numeric

    rfc = (pat[:2] + mat[0] + nombre[0] + anio[2:] + mes + dia)
    #Genera el RFC con las reglas básicas
    #Generates RFC using basic rule
    if self.modos.get() == "Pilas":
        self.datos.append(rfc)
    else:
        self.datos.insert(0, rfc)
    #Inserta en PILA (al final) o en COLA (al inicio)
    #Inserts in STACK (end) or QUEUE (beginning)

    self.actualizar_lista()
    #Actualiza la lista de la interfaz
    #Refreshes the visual list
    messagebox.showinfo("RFC agregado", f"RFC generado: {rfc}")
    # Muestra RFC generado/Shows generated RFC

    '''Limpiar campos'''
    self.nombre.delete(0, END)
    self.Pat.delete(0, END)
    self.Mat.delete(0, END)
    self.Dia.delete(0, END)
    self.Mes.delete(0, END)
    self.An.delete(0, END)
```

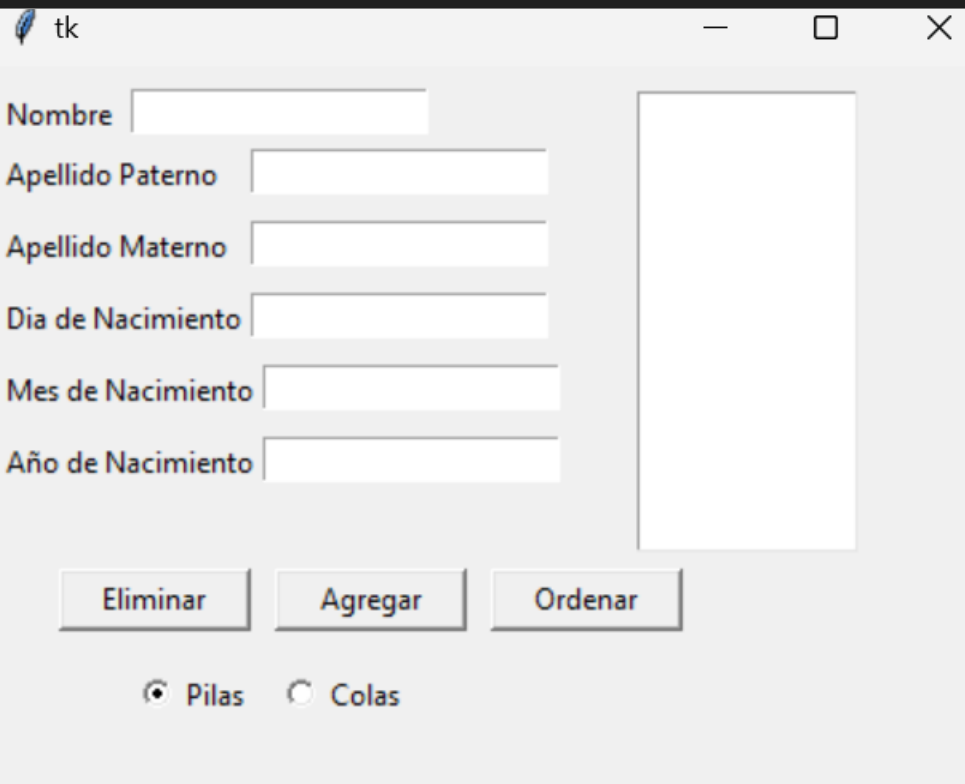
```
def actualizar_lista(self):
    self.lista.delete(0, END)
    # Elimina todos los elementos de la lista visual
    # Deletes all items from listbox
    for i in self.datos:
        self.lista.insert(END, i)
    # Inserta cada dato en la lista visual
    # Inserts each stored RFC in the listbox

def eliminard(self):
    if not self.datos:
        messagebox.showwarning("Atención", "No hay elementos para eliminar.")
        return
    # Si no hay elementos en la lista
    # If the list is empty
    if self.modos.get() == "Pilas":
        # Elimina según PILA o COLA
        # Removes according to STACK or QUEUE mode
        self.datos.pop() # Elimina el último
    else:
        self.datos.pop(0) # Elimina el primero

    self.actualizar_lista()
    # Actualiza la lista visual
    # Updates list display
```

```
def ordenar(self):
    if self.lista.size() <= 0:
        messagebox.showerror("Error", "La lista esta vacia")
        return
    # Verifica si la lista está vacía
    # Checks if list is empty
    if self.modos.get() == 'Pilas':
        # Este método NO ordena: borra elementos según PILA o COLA
        # This method does NOT sort: it deletes using STACK or QUEUE logic
        # Último que entra, primero que sale
        self.lista.delete(self.lista.size()-1)
    else:
        # Primero que entra, primero que sale
        self.lista.delete(0)
    self.label.config(text=f'Elementos en la lista: {str(self.lista.size())}')
    # Actualiza el contador en etiqueta
    # Updates label counter
```

```
if __name__ == '__main__':
    app = Principal()
    app.inicio()
```



PRACTICA 3

Agregar Nombre, Teléfono y Domicilio a una lista y agregar su sexo con un RadioButton

```
'''Nombre, telefono, domicilio, validar letras en nombre y numeros en telefono,
se agregan en la lista contando su sexo (F/M) y se agrega su clave'''
from tkinter import *
from tkinter import messagebox
from validacionesp2 import Validar
import numpy as np

'''VENTANA, TAMAÑO'''
class Principal():
    def __init__(self):
        self.val = Validar()
        self.ven = Tk()
        ancho = 350
        alto = 250
        ventana_alto = self.ven.winfo_screenwidth()
        ventana_ancho = self.ven.winfo_screenheight()
        x = (ventana_alto // 2) - (ancho // 2)
        y = (ventana_ancho // 2) - (alto // 2)
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")

    def quitar_placeholder1(self,event):
        if self.nombre.get() == self.placeholder1:
            self.nombre.delete(0,END)
            self.nombre.config(fg="black")
    def quitar_placeholder2(self,event):
        if self.telefono.get() == self.placeholder2:
            self.telefono.delete(0,END)
            self.telefono.config(fg="black")
    def quitar_placeholder3(self,event):
        if self.domicilio.get() == self.placeholder3:
            self.domicilio.delete(0,END)
            self.domicilio.config(fg="black")

    #Si el campo nombre contiene el placeholder, lo borra y pone texto negro
    #If the name field contains the placeholder, delete it and set black text

    def poner_placeholder1(self,event):
        if self.nombre.get() == "":
            self.nombre.insert(0, self.placeholder1)
            self.nombre.config(fg="gray")
    def poner_placeholder2(self,event):
        if self.telefono.get() == "":
            self.telefono.insert(0, self.placeholder2)
            self.telefono.config(fg="gray")
    def poner_placeholder3(self,event):
        if self.domicilio.get() == "":
            self.domicilio.insert(0, self.placeholder3)
            self.domicilio.config(fg="gray")
```

```
'''INTERFAZ DE LA VENTANA'''
def inicio(self):
    #CAJA DE TEXTO NOMBRE
    self.placeholder1="Nombre"
    self.nombre = Entry(self.ven, fg="gray")
    self.nombre.insert(0, self.placeholder1)
    self.nombre.bind("<FocusIn>", self.quitar_placeholder1)
    self.nombre.bind("<FocusOut>", self.poner_placeholder1)
    #self.nombre.bind("<Return>", self.validarCaja)
    self.nombre.place(x=10, y=10, width=100)
    #CAJA DE TEXTO TELEFONO
    self.placeholder2="Telefono"
    self.telefono = Entry(self.ven, fg="gray")
    self.telefono.insert(0, self.placeholder2)
    self.telefono.bind("<FocusIn>", self.quitar_placeholder2)
    self.telefono.bind("<FocusOut>", self.poner_placeholder2)
    #self.telefono.bind("<Return>", self.validarCaja)
    self.telefono.place(x=120, y=10, width=100)
    #CAJA DE TEXTO DOMICILIO
    self.placeholder3="Domicilio"
    self.domicilio = Entry(self.ven, fg="gray")
    self.domicilio.insert(0, self.placeholder3)
    self.domicilio.bind("<FocusIn>", self.quitar_placeholder3)
    self.domicilio.bind("<FocusOut>", self.poner_placeholder3)
    self.domicilio.bind("<Return>", self.validarCaja)
    self.domicilio.place(x=230, y=10, width=100)

    Label(self.ven, text= "Sexo").place(x=10, y=30)
    self.modos = StringVar(value="F")
    Radiobutton(self.ven, text="F", variable=self.modos, value="F").place(x=10, y=50)
    Radiobutton(self.ven, text="M", variable=self.modos, value="M").place(x=10, y=70)
    self.lista= Listbox(self.ven, height=6, width=35, bg="white", font=("Helvetica",10))
    self.lista.place(x=10, y=100)
    Button(self.ven, text="Agregar", command=self.validarCaja, width=10).place(x=80, y=50,
                                                                                   width=100, height=30)

    self.ven.mainloop()

def agregar(self):
    pass
```

```
def validarCaja(self, event=0):
    if(self.nombre.get == self.placeholder1
       or self.telefono.get == self.placeholder2
       or self.domicilio.get == self.placeholder3 or self.domicilio.get()==""):
        messagebox.showerror('Error', 'Faltan datos')
    #Validar si los campos están vacíos o siguen con el placeholder
    #Validate if fields are empty or still contain the placeholder
    else:
        nombre = self.nombre.get()
        telefono = self.telefono.get()
        domicilio = self.domicilio.get()
        #Obtener valores reales de los campos
        #Get actual field values

        if not self.val.ValidarLetra(nombre):
            messagebox.showerror('Error', 'El nombre solo debe contener letras')
            return
        #Validar que sean letras/Validate only letters
        if not self.val.ValidarNumeros(telefono):
            messagebox.showerror('Error', 'El teléfono solo debe contener números')
            return
        #Validar que el teléfono tenga solo números
        #Validate phone number contains only digits
        if self.modos.get() == "F":
            sexo = "Femenino"
        #Convertir valor del radiobutton a texto
        #Convert radiobutton value to readable text

        else:
            sexo = "Masculino"
        clave = nombre[0] + telefono[0] + domicilio[0:2]
        #Crear clave usando inicial/Create ID using initials
        persona = clave + "-" + nombre + "-" + telefono + "-" + domicilio + "-" + sexo
        #Armar cadena final/Build final record string
        self.lista.insert(self.lista.size()+1,persona)
        #Insertar en la lista/Insert into listbox

if __name__=='__main__':
    app= Principal()
    app.inicio()

'''Validar nombre y telefono'''
```

tk

Nombre Telefono Domicilio

Sexo

☒ F ☐ M

Agregar

PRACTICA 4

Agregar Nombre, Edad y Correo a la tabla generando una Clave, al seleccionar podrás editar el registro

```
'''Nombre, edad y correo y agregar validiendo letras y numeros,  
se muestran en la tabla con su clave, al seleccionar un registro de la tabla  
se pueda modificar'''
```

```
from tkinter import *  
from tkinter import messagebox  
from validacionesp2 import Validar  
import numpy as np  
from tkinter import ttk  
import random  
  
'''VENTANA, TAMAÑO'''  
class Principal():  
    def __init__(self):  
        self.val = Validar()  
        self.ven = Tk()  
        ancho = 500  
        alto = 300  
        ventana_alto = self.ven.winfo_screenwidth()  
        ventana_ancho = self.ven.winfo_screenheight()  
        x = (ventana_alto // 2) - (ancho // 2)  
        y = (ventana_ancho // 2) - (alto // 2)  
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")  
        self.cont = 0  
        self.bandera = False  
        self.renglon = -1  
        self.index = 0
```

```
'''INTERFAZ DE VENTANA'''  
def inicio(self):  
  
    Label(self.ven, text= "Nombre").place(x=10, y=10)  
    self.nombre = Entry(self.ven, fg="blue")  
    self.nombre.place(x=10, y=40, width=100)  
  
    Label(self.ven, text= "Edad").place(x=130, y=10)  
    self.edad = Entry(self.ven, fg="green")  
    self.edad.place(x=125, y=40, width=100)  
  
    Label(self.ven, text= "Correo").place(x=250, y=10)  
    self.correo = Entry(self.ven, fg="purple")  
    self.correo.place(x=240, y=40, width=100)  
  
    Button(self.ven, text="Agregar", command=self.agregarElemento, width=10).place(x=380, y=50,  
                                                                                    width=100, height=30)  
    Button(self.ven, text="Eliminar", command=self.Eliminar, width=10).place(x=380, y=90,  
                                                                              width=100, height=30)  
    Button(self.ven, text="Seleccionar", command=self.validarCaja, width=10).place(x=380, y=130,  
                                                                                  width=100, height=30)  
  
    #DATAGRID  
    columnas = ("Clave", "Nombre", "Correo", "Edad")  
    self.tabla = ttk.Treeview(self.ven, columns = columnas, show = "headings")  
    self.tabla.place(x=10, y=100, width=350, height=190)
```

```
for col in columnas:
    self.tabla.heading(col, text=col)
    self.tabla.column(col, anchor="center", width=30)
scrolly = ttk.Scrollbar(self.ven, orient="vertical", command= self.tabla.yview)
scrollx = ttk.Scrollbar(self.ven, orient="horizontal",command= self.tabla.xview)
scrolly.place(x=360, y=90,height=200)
scrollx.place(x=10, y=280, width=350)

self.ven.mainloop()
```

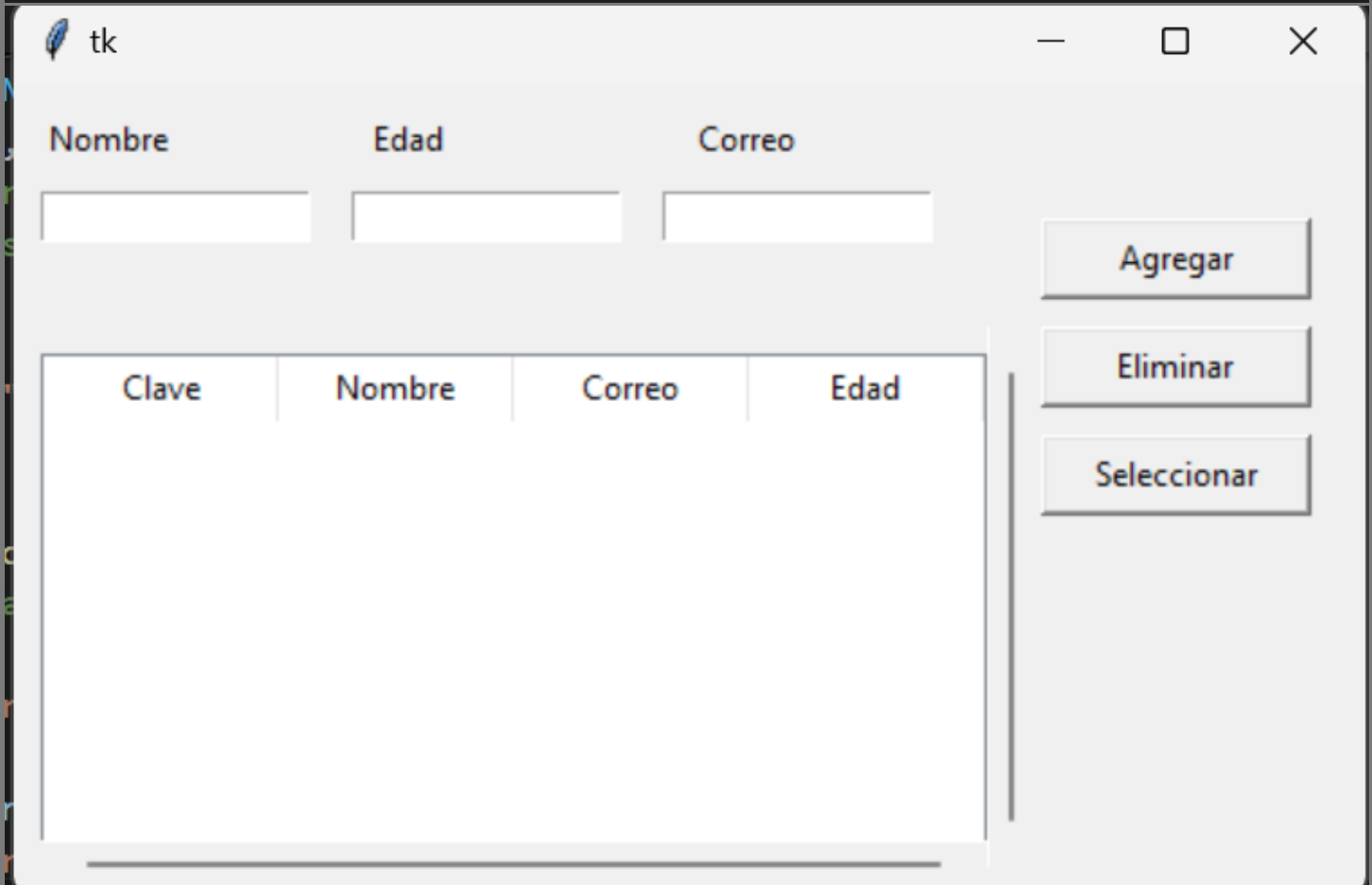
```
def validarCaja(self):
    self.renglon = self.tabla.selection()
    #Obtener la fila seleccionada de la tabla
    #Get the selected row from the table
    if not self.renglon:
        messagebox.showerror("Error","Elije una fila")
    #Validar si no se seleccionó nada
    #Validate if nothing was selected
    else:
        valores= self.tabla.item(self.renglon, "values")
        print(valores)
        #Obtener valores de la fila seleccionada
        #Get values from selected row
        self.index = valores[0]
        self.index =self.index[:len(self.index)-2]
        print(self.index)
        #Extraer la clave sin los últimos dos caracteres
        #Extract the key without the last two characters
        self.nombre.insert(0,valores[1])
        self.edad.insert(0,valores[3])
        self.correo.insert(0,valores[2])
        #Cargar los datos en las cajas de texto
        #Load row data into entry boxes
        self.bandera = True
        #Activar modo edición
        #Activate edit mode
```

```
def agregarElemento(self):
    if(len(self.nombre.get())==0 or len(self.edad.get())==0 or len(self.correo.get())==0):
        messagebox.showerror("Error","Faltan Datos")
    #Validar que no haya campos vacíos/Validate fields are not empty
    else:
        nombre = self.nombre.get()
        edad =self.edad.get()
        correo = self.correo.get()
        if self.bandera == False:
            #Contador que aumenta cada registro/Global row counter
            self.cont += 1
            clave = str(self.cont)+str(random.randint(1,100))+self.nombre.get()[0:2].upper()
            #Generar clave: número + random + 2 letras del nombre
            #Generate key: number + random + first 2 letters of name
            self.tabla.insert("", "end", values=(clave,nombre,correo,edad))
            #Insertar fila nueva/Insert new row
            self.nombre.delete(0,END)
            self.edad.delete(0,END)
            self.correo.delete(0,END)
            #Limpiar campos/Clear fields

        else:
            clave = self.index+self.nombre.get()[0:2]
            #Generar la nueva clave con las iniciales del nombre
            #Generate edited key with new initials
            print("Modo de edicion activado")
            self.tabla.item(self.renglon,value = (clave, nombre, edad, correo))
            #Actualizar la fila seleccionada
            #Update selected row
            self.nombre.delete(0,END)
            self.edad.delete(0,END)
            self.correo.delete(0,END)
            #Limpiar entradas y restaurar modo agregar
            #Clear fields and restore add mode
            self.bandera = False
            self.renglon = -1
            messagebox.showinfo("Correcto","Datos Actualizados")
```

```
def Eliminar(self):
    renglon = self.tabla.selection()
    #Obtener la fila seleccionada/Get selected row
    if not renglon:
        messagebox.showerror("Error", "Elige una fila")
    else:
        self.tabla.delete(renglon)
        messagebox.showinfo("Correcto", "Fila Eliminada")
        #Eliminar la fila/Delete row

if __name__ == '__main__':
    app = Principal()
    app.inicio()
```



tk

Nombre Edad Correo

Clave	Nombre	Correo	Edad
-------	--------	--------	------

Agregar

Eliminar

Seleccionar

PARCIAL 4

PRACTICA 1

Escribir 2 números, botón enviar para ir a otra ventana y sumar los dos números ingresados

```
#LIBRERIAS/LIBRARIES
from tkinter import *
from tkinter import messagebox

#CLASE PRINCIPAL/MAIN CLASS
class Principal():

    #TAMAÑO DE LA VENTANA /WINDOW SIZE
    def __init__(self, master):
        self.vetana = master
        self.vetana.title("Practica 1 Parcial 3")
        ancho_vetana = 250
        alto_vetana = 200
        ancho_pantalla = self.vetana.winfo_screenwidth()
        alto_pantalla = self.vetana.winfo_screenheight()
        x = (ancho_pantalla // 2) - (ancho_vetana // 2)
        y = (alto_pantalla // 2) - (alto_vetana // 2)
        self.vetana.geometry(f"{ancho_vetana}x{alto_vetana}+{x}+{y}")

#INTERFAZ DE LA VENTANA/WINDOW INTERFACE
def inicio(self):
    Label(self.vetana, text = "Escribe un numero: ").place(x = 20, y = 20)
    self.n1 = Entry(self.vetana)
    self.n1.place(x = 50, y = 50)

    Label(self.vetana, text= "Escribe un numero ").place(x = 20, y = 75)
    self.n2 = Entry(self.vetana)
    self.n2.place(x = 50, y = 100)

    Button(self.vetana, text = "Enviar", width=15, command= self.enviar).place(x = 50, y = 130)
    Button(self.vetana, text = "Cerrar", width=15, command= self.cerrar).place(x = 50, y = 160)

    self.vetana.mainloop()
```

```
def enviar(self):
    try:
        #OBTIENE LOS VALORES DE LAS CAJAS DE TEXTO
        #GET THE VALUES FROM THE TEXTBOXES
        "SOLO PERMITE NUMEROS POR (INT)"
        caja1 = int(self.n1.get())
        caja2 = int(self.n2.get())
        #LIMPIA LAS CAJAS
        #CLEARS THE TEXTBOXES
        self.n1.delete(0, END)
        self.n2.delete(0, END)
        #OCULTA VENTANA 1 (PRINCIPAL)
        #HIDES MAIN WINDOW
        self.vetana.withdraw()
        #CREACION VENTANA 2
        #CREATES WINDOW 2
        otra = Toplevel(self.vetana)
        #ABRE LA CLASE DE LA VENTANA 2
        #OPENS THE CLASS FOR WINDOW 2
        Ventana2(otra, self.vetana, caja1, caja2)
```

```
except ValueError:
    messagebox.showerror("Error", "Algun dato no es numero")

    self.n1.delete(0, END)
    self.n2.delete(0, END)
```

```
#CIERRA COMPLETAMENTE VENTANA 1 AL PRESIONAR BOTON SALIR
#CLOSES MAIN WINDOW COMPLETELY WHEN PRESSING EXIT BUTTON
def cerrar(self):
    self.vetana.destroy()
```

```
#VENTANA 2/WINDOW 2
class Ventana2 ():
    def __init__(self, master, vetana, c1, c2):
        self.venDos = master
        self.venDos.title("Practica 1 Parcial 3")
        ancho_vetana = 250
        alto_vetana = 200
        ancho_pantalla = self.venDos.winfo_screenwidth()
        alto_pantalla = self.venDos.winfo_screenheight()
        x = (ancho_pantalla // 2) - (ancho_vetana // 2)
        y = (alto_pantalla // 2) - (alto_vetana // 2)
        self.venDos.geometry(f"{ancho_vetana}x{alto_vetana}+{x}+{y}")

        Label(self.venDos, text = "Hola mundo").place(x = 50, y = 20)

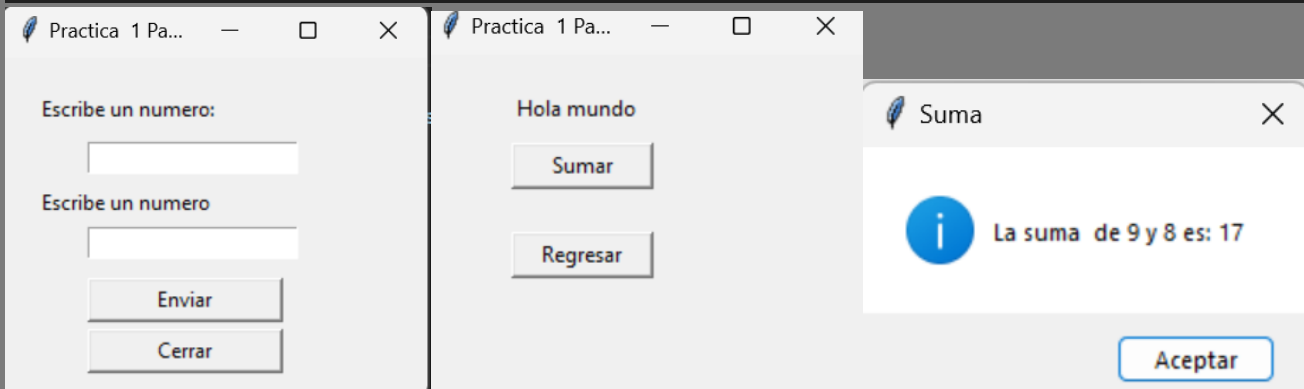
        Button(self.venDos, text = "Regresar", width=10, command= self.regresar).place(x = 50, y = 100)
        Button(self.venDos, text = "Sumar", width=10, command= self.sumar).place(x = 50, y = 50)
        #GUARDA LOS VALORES ENVIADOS
        #SAVES THE RECEIVED VALUES
        self.vetana = vetana
        self.c1 = c1
        self.c2 = c2

    def regresar(self):
        #CIERRA VENTANA 2/CLOSES WINDOW 2
        self.venDos.destroy()
        #VUELVE A MOSTRAR VENTANA 1/SHOWS WINDOW 1 AGAIN
        self.vetana.deiconify()

        #SUMA DE LOS DOS NUMEROS QUE MUESTRA EN UN MENSAJE
        #SUMS THE TWO NUMBERS AND SHOWS THEM IN A MESSAGEBOX

    def sumar(self):
        messagebox.showinfo("Suma", f"La suma de {self.c1} y {self.c2} es: {self.c1 + self.c2}")

if __name__ == "__main__":
    master = Tk()
    app = Principal(master)
    app.inicio()
    master.mainloop()
```



PRACTICA 2

Ingresar Usuario y password, botón para ir a otra ventana y a través de una base de datos guarda los usuarios ingresados puedes: modificar, eliminar y agregar

```
#LIBRERIAS/LIBRARIES
from tkinter import *
from tkinter import messagebox
from tkinter import ttk
import tkinter as tk
import sqlite3

#CREACION BASE DE DATOS/DATABASE CREATION
def crearBaseDatos():
    #CONECTAR BASE DE DATOS LLAMDA Usuarios.db
    #CONNECT TO DATABASE NAMED Usuarios.db
    con = sqlite3.connect("Usuarios.db")
    cursor = con.cursor()
    #CREA LA TABLA
    #CREATE THE TABLE
    cursor.execute("""CREATE TABLE IF NOT EXISTS usuarios(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        usuario TEXT NOT NULL,password TEXT NOT NULL)""")
    #VERIFICA SI EXISTE EL USUARIO "admin"
    #CHECK IF USER "admin" EXISTS
    cursor.execute("SELECT * FROM usuarios WHERE usuario='admin'")
    #FETOCHES DEVUELVE REGISTRO NONE, SI NO EXISTE ENTRA AL IF
    #FETCHONE RETURNS NONE IF NO RECORD EXISTS, THEN ENTER IF
    if not cursor.fetchone():
        #INSERTA USUARIO POR DEFECTO/INSERT DEFAULT USER
        cursor.execute("INSERT INTO usuarios (usuario,password) VALUES (?,?)",("admin","12345"))
    #GUARDA CAMBIOS Y CERRAR/SAVE CHANGES AND CLOSE
    con.commit()
    con.close()

#CLASE VENTANA 1/WINDOW 1 CLASS
class Ventana():
    #TAMAÑO DE VENTANA/WINDOW SIZE
    def __init__(self,master):
        self.ven = master
        self.ven.title('Programa 2')
        ancho = 250
        alto = 200
        ventana_ancho = self.ven.winfo_screenwidth()
        ventana_alto = self.ven.winfo_screenheight()
        x = (ventana_ancho // 2) - (ancho // 2)
        y = (ventana_alto // 2) - (alto // 2)
        self.ven.geometry(f'{ancho}x{alto}+{x}+{y}')
```

```
#INTERFAZ VENTANA 1/WINDOW 1 INTERFACE
def Inicio(self):
    Label(self.ven,text='Usuario').place(x=50,y=20)
    self.n1 = Entry(self.ven)
    self.n1.place(x=50,y=50)

    Label(self.ven,text='Password').place(x=50,y=75)
    self.n2 = Entry(self.ven, show="*")
    self.n2.place(x=50,y=100)

    Button(self.ven,text='Validar',command=self.Enviar,width=10,fg='green').place(x=30,y=140)
    Button(self.ven,text='Cerrar',command=self.Cerrar,width=10,fg='red').place(x=150,y=140)

def Enviar(self):
    #OBTIENE LOS DATOS DE LAS CAJAS DE TEXTO
    #GET DATA FROM TEXTBOXES
    u = self.n1.get()
    p = self.n2.get()
    #CONECTA CON LA BASE DE DATOS/CONNECT TO DATABASE
    con = sqlite3.connect("Usuarios.db")
    cursor = con.cursor()
    #VERIFICA SI EXISTE EL USUARIO/CHECK IF USER EXISTS
    cursor.execute("SELECT * FROM usuarios WHERE usuario=? and password=?", (u,p))
    #OBTIENE EL RESULTADO/GET RESULT
    resultado = cursor.fetchone()
    #CIERRA LA CONEXION/CLOSE DATABASE CONNECTION
    con.close()
    #SI EL RESULTADO SE ENCUENTRA ENTONCES..
    #IF RESULT FOUND THEN...
    if resultado:
        #LIMPIA CAJAS DE TEXTO/CLEAR TEXTBOXES
        self.n1.delete(0,END)
        self.n2.delete(0,END)
        #OCULTA LA VENTANA 1/HIDE WINDOW 1
        self.ven.withdraw()
        #CREA VENTANA 2/CREATE WINDOW 2
        otra = Toplevel(self.ven)
        #ABRE LA VENTANA 2/OPEN WINDOW 2
        Ventanados(otra,self.ven,u)
```

```
else:
    messagebox.showerror('Error','Datos incorrectos')
    self.n1.delete(0,END)
    self.n2.delete(0,END)
```

```
#CIERRA TOTALMENTE LAS VENTANAS CON BOTON SALIR
```

```
#COMPLETELY CLOSE WINDOWS WITH EXIT BUTTON
```

```
def Cerrar(self):
    self.ven.destroy()
```

```
#CLASE VENTANA 2/WINDOW 2 CLASS
```

```
class Ventanados():
```

```
    #TAMAÑO VENTANA 2/WINDOW 2 SIZE
```

```
    def __init__(self, master, ven, u):
```

```
        #VENTANA 1/WINDOW 1
```

```
        self.ven = ven
```

```
        #NOMBRE USUARIO QUE INICIO SESION
```

```
        #USERNAME THAT LOGGED IN
```

```
        self.usuario = u
```

```
        #VENTANA 2/WINDOW 2
```

```
        self.dos = master
```

```
    #TAMAÑO DE VENTANA 2/WINDOW 2 SIZE
```

```
    self.dos.title('Programa 1')
```

```
    ancho = 500
```

```
    alto = 320
```

```
    ventana_ancho = self.dos.winfo_screenwidth()
```

```
    ventana_alto = self.dos.winfo_screenheight()
```

```
    x = (ventana_ancho // 2) - (ancho // 2)
```

```
    y = (ventana_alto // 2) - (alto // 2)
```

```
    self.dos.geometry(f'{ancho}x{alto}+{x}+{y}')
```

```
#INTERFAZ DE VENTANA 2/WINDOW 2 INTERFACE
```

```
Button(self.dos, text='Regresar', command=self.Regresar, width=10).place(x=150, y=140)
```

```
self.us = Label(self.dos, text=f'Bienvenido \n {self.usuario}')
```

```
self.us.place(x=400, y=1)
```

```
self.Mostrar()
```

```
self.mostrarTabla()
```

```
#CREACION DEL MENU/MENU CREATION
self.menus = tk.Menu(self.dos)
self.dos.config(menu=self.menus)
#SUBMENU "ARCHIVO"/SUBMENU "ARCHIVO"
self.archivos = tk.Menu(self.menus,tearoff=0)
self.archivos.add_command(label='Agregar',command=self.Crearusuario)
self.indexAgregar = self.archivos.index("end")
self.archivos.add_command(label='Modificar',command=self.Modificarusr)
self.indexModificar = self.archivos.index("end")
self.archivos.add_command(label='Eliminar',command=self.Eliminarusr)
self.indexEliminar = self.archivos.index("end")
self.archivos.add_command(label='Salir',command=self.salir)
self.menus.add_cascade(label='Archivo',menu=self.archivos)
#INHABILITA LAS OPCIONES AGREGAR, MODIFICAR Y ELIMINAR
#DISABLE ADD, MODIFY AND DELETE OPTIONS
self.archivos.entryconfig(self.indexAgregar,state = 'disable')
self.archivos.entryconfig(self.indexEliminar,state = 'disable')
self.archivos.entryconfig(self.indexModificar,state = 'disable')
#ACTIVA OPCIONES DEPENDIENDO DEL USUARIO
#ENABLE OPTIONS DEPENDING ON USER
self.roles()
def roles(self):
    #SI EL USUARIO ES admin PERMISO TOTAL, TODO EL MENU ACTIVADO
    #IF USER IS admin, FULL PERMISSION, ALL MENU ENABLED
    if self.usuario == 'admin':
        self.archivos.entryconfig(self.indexAgregar,state='normal')
        self.archivos.entryconfig(self.indexEliminar,state='normal')
        self.archivos.entryconfig(self.indexModificar,state='normal')
    #SI EL USUARIO ES supervisor PUEDE AGREGAR SOLAMENTE
    #IF USER IS supervisor CAN ONLY ADD
    elif self.usuario == 'Supervisor' or self.usuario == 'supervisor':
        self.archivos.entryconfig(self.indexAgregar,state='normal')
        self.archivos.entryconfig(self.indexEliminar,state='disable')
        self.archivos.entryconfig(self.indexModificar,state='disable')
    #SI EL USUARIO ES jefe de area PUEDE ELIMINAR SOLAMENTE
    #IF USER IS jefe de area CAN ONLY DELETE
    elif self.usuario == 'Jefe de area' or self.usuario == 'jefe de area':
        self.archivos.entryconfig(self.indexAgregar,state='disable')
        self.archivos.entryconfig(self.indexEliminar,state='normal')
        self.archivos.entryconfig(self.indexModificar,state='disable')
```



```
def Crearusuario(self):
    #VERIFICA CAJAS NO VACIAS/CHECK NON-EMPTY TEXTBOXES
    if len(self.usu.get()) != 0 and len(self.pas.get()) != 0:
        #CONECTA A LA BASE DE DATOS/CONNECT TO DATABASE
        con = sqlite3.connect("Usuarios.db")
        cursor = con.cursor()
        #INSERTA LOS DATOS A LA TABLA (USUARIO Y CONTRASEÑA)
        #INSERT DATA INTO TABLE (USER AND PASSWORD)
        cursor.execute("INSERT INTO usuarios (usuario,password) VALUES (?,?)", (self.usu.get(), self.pas.get()))
        con.commit()
        con.close()
        self.Actualizartabla()
        self.borrarcaja('Usuario agregado correctamente')
    else:
        messagebox.showerror('Error','Faltan Datos')
```

```
def Seleccionfila(self, event):
    #FILA SELECCIONADA/#ELECTED ROW
    try:
        index = self.tabla.selection()[0]
    except:
        return
    #OBTIENE LOS VALORES DE ESA FILA
    #GET VALUES FROM THAT ROW
    valores = self.tabla.item(index,"values")
    #LIMPIA CAJAS DE TEXTO/CLEAR TEXTBOXES
    self.usu.delete(0,END)
    self.pas.delete(0,END)
    #INSERTA LOS VALORES DEL REGSITRO SELECCIONADO
    #INSERT SELECTED RECORD VALUES
    self.usu.insert(0,valores[1])
    self.pas.insert(0,valores[2])
```

```
def Actualizartabla(self):
    #LIMPIA TODA LA TABLA/CLEAR ENTIRE TABLE
    for i in self.tabla.get_children():
        self.tabla.delete(i)
    #VUELVE A LLENAR LA TABLA/REFILL TABLE
    self.mostrarTabla()
```

```
def Modificarus(self):
    #VERIFICA QUE HAYA SELECCIONADO UN REGISTRO
    #CHECK IF A RECORD IS SELECTED
    try:
        index = self.tabla.selection()[0]
    except:
        messagebox.showerror('Error', 'Elige un usuario')
        return
    #OBTIENE LOS VALORES DE ESA FILA
    #GET VALUES FROM THAT ROW
    valores = self.tabla.item(index, "values")
    id = valores[0]
    #CAJAS NO VACIAS/TEXTBOXES NOT EMPTY
    if len(self.usu.get()) != 0 and len(self.pas.get()) != 0:
        #REALIZA LA MODIFICACION EN BASE DE DATOS
        #UPDATE RECORD IN DATABASE
        u = self.usu.get()
        p = self.pas.get()
        #CONECTA A LA BASE DE DATOS/CONNECT TO DATABASE
        con = sqlite3.connect("Usuarios.db")
        cursor = con.cursor()
        cursor.execute("UPDATE usuarios SET usuario=?, password=? WHERE id=?", (u, p, id))
        con.commit()
        con.close()
        self.Actualizartabla()
        self.borrarcaja('Usuario modificado correctamente')
    else:
        messagebox.showerror('Error', 'Faltan datos')

def Eliminarusr(self):
    try:
        #OBTIENE LA FILA SELECCIONADA DE LA TABLA
        #GET SELECTED ROW
        index = self.tabla.selection()[0]
        #EXTRAER LOS VALORES DE LA FILA/EXTRACT VALUES FROM ROW
        valores = self.tabla.item(index, "values")
        #TOMA EL ID DEL USUARIO EN LA BASE DE DATOS
        #GET ID OF USER IN DATABASE
        id = int(valores[0])
        #NOMBRE DEL USUARIO/USERNAME
        usuario = valores[1]
        #BLOQUEA QUE EL USUARIO SE ELIMINE A SI MISMO
        #PREVENT USER FROM DELETING THEMSELVES
        if usuario == self.usuario:
            self.borrarcaja()
            messagebox.showerror('Error', 'No puedes eliminar tu propio usuario')
```

```
else:
    #SI NO ES EL USUARIO ACTUAL, PROCEDE LA ELIMINACION
    #IF NOT CURRENT USER, PROCEED TO DELETE
    con = sqlite3.connect("Usuarios.db")
    cursor = con.cursor()
    #ELIMINA EL USUARIO DE LA BASE
    #DELETE USER FROM DATABASE
    cursor.execute("DELETE FROM usuarios WHERE id=?", (id,))
    #REORGANIZA LOS ID PARA QUE NO QUEDEN HUECOS
    #REORGANIZE IDS SO THERE ARE NO GAPS
    cursor.execute("UPDATE usuarios SET id = id - 1 WHERE id > ?", (id,))
    #REINICIA EL CONTADOR DE AUTOINCREMENT
    #RESET AUTOINCREMENT COUNTER
    cursor.execute("DELETE FROM sqlite_sequence WHERE name='usuarios'")
    con.commit()
    con.close()
    self.Actualizartabla()
    self.borrarcaja('Usuario borrado correctamente')
```

```
except:
    messagebox.showerror('Error', 'Elige un usuario')
```

```
#SALIR COMPLETAMENTE DE LAS DOS VENTANAS
```

```
#EXIT COMPLETELY FROM BOTH WINDOWS
```

```
def salir(self):
```

```
    self.dos.destroy()
```

```
    self.ven.destroy()
```

```
def mostrarTabla(self):
```

```
    con = sqlite3.connect("Usuarios.db")
```

```
    cursor = con.cursor()
```

```
    #SELECCIONA TODOS LOS REGISTROS
```

```
    #SELECT ALL RECORDS
```

```
    cursor.execute("SELECT * FROM usuarios")
```

```
    #INSERTA LOS DATOS EN EL TREEVIEW
```

```
    #INSERT DATA INTO TREEVIEW
```

```
    for i in cursor.fetchall():
```

```
        self.tabla.insert("", END, values=i)
```

```
    #CIERRA LA BASE DE DATOS
```

```
    #CLOSE DATABASE
```

```
    con.close()
```

```
def Mostrar(self):
    #CREA COLUMNAS PARA LA TABLA
    #CREATE COLUMNS FOR TABLE
    columnas = ("ID","USUARIO","PASSWORD")
    self.tabla = ttk.Treeview(self.dos,columns=columnas,show='headings')
    self.tabla.place(x=10,y=100,width=350,height=190)
    #POSICIONAR LA TABLA Y CONFIGURAR LAS COLUMNAS
    #POSITION THE TABLE AND CONFIGURE COLUMNS
    for col in columnas:
        self.tabla.heading(col,text=col)
        self.tabla.column(col,anchor='center',width=30)
    #CREA BARRAS DE DESPLAZAMIENTO/CREATE SCROLLBARS
    scrolly = ttk.Scrollbar(self.dos,orient='vertical',command=self.tabla.yview)
    scrollx = ttk.Scrollbar(self.dos,orient='horizontal',command=self.tabla.xview)
    scrolly.place(x=360,y=90,height=200)
    scrollx.place(x=10,y=280,width=350)

    Label(self.dos,text='Escribe el usuario').place(x=10,y=10)
    self.usu = Entry(self.dos)
    self.usu.place(x=10,y=30)

    Label(self.dos,text='Escribe el password').place(x=150,y=10)
    self.pas = Entry(self.dos)
    self.pas.place(x=150,y=30)
    #VINCULAR UN EVENTO AL SELECCIONAR UNA FILA
    self.tabla.bind("<<TreeviewSelect>>",self.Seleccionfila)

def borrarcaja(self,mensaje=''):
    self.usu.delete(0,END)
    self.pas.delete(0,END)
    if mensaje:
        messagebox.showinfo('Listo',mensaje)

def Regresar(self):
    #CIERRA VENTANA 2/CLOSE WINDOW 2
    self.dos.destroy()
    #MUESTRA NUEVAMENTE LA VENTANA 1/SHOW WINDOW 1 AGAIN
    self.ven.deiconify()

if __name__ == '__main__':
    crearBaseDatos()
    master = Tk()
    app = Ventana(master)
    app.Inicio()
    master.mainloop()
```



Practica 2 Parcial 3



CRUD DE PRODUCTOS

Producto	Descripcion	Precio	Cantidad

ID	CODIGO	PRODUCTO	PRECIO	DESCRIPCION	STOCK
2	JA9P	jabon	98.0	polvo	2
3	LE46D	leche	60.0	deslactosada	2

Agregar

Modificar

Eliminar

Practica4.py

usuarios.db X

usuarios.db



Filter 2 tables...

Rows: 1

Filter 1 rows...

Upgrade to PRO

TABLES

> sqlite_sequence

> usuarios

name



seq



Filtrar



Filtrar



1

usuarios

4



2

PRACTICA 3

**Ingresar un número que desee y envía a otra ventana el
número y sus repeticiones**

```
'''
```

```
hacer un programa en tkinter que en una ventana mediante  
una caja de texto lea un numero, ese numero se enviara a  
otra ventana donde en un lisbiu, mostrara ese numero, el numero de veces  
'''
```

```
from tkinter import *  
from tkinter import messagebox  
from tkinter import ttk
```

```
class Principal():  
    def __init__(self, master):  
        self.vetana = master  
        self.vetana.title("Practica 1 Parcial 3")  
        ancho_vetana = 250  
        alto_vetana = 200  
        ancho_pantalla = self.vetana.winfo_screenwidth()  
        alto_pantalla = self.vetana.winfo_screenheight()  
        x = (ancho_pantalla // 2) - (ancho_vetana // 2)  
        y = (alto_pantalla // 2) - (alto_vetana // 2)  
        self.vetana.geometry(f"{ancho_vetana}x{alto_vetana}+{x}+{y}")  
  
    def inicio(self):  
        Label(self.vetana, text = "Numero: ").place(x = 100, y = 50)  
        self.nume1 = Entry(self.vetana)  
        self.nume1.place(x = 60, y = 100)  
        Button(self.vetana, text = "Enviar", command = self.enviar).place(x = 100, y = 150)  
        Button(self.vetana, text = "Salir", command = self.destruir).place(x = 200, y = 150)  
  
    def enviar(self):  
        try:  
            if len(self.nume1.get()) <= 0:  
                messagebox.showerror("Error", "Campo vacio")  
                return 1  
  
            numero = int(self.nume1.get())  
            self.nume1.delete(0, END)  
            self.vetana.withdraw()  
            nuevaVen = Toplevel(self.vetana)  
            Ventana_lista(nuevaVen, self.vetana, numero)  
  
        except ValueError:  
            messagebox.showerror("Error", "No son numeros")  
            self  
  
    def destruir(self):  
        self.vetana.destroy()
```

```

class Ventana_lista():
    def __init__(self, master, vetana, dato):
        self.ventaPrimaria = vetana
        self.numero = dato
        self.vetnanaSec = master
        self.vetnanaSec.title("Resultado de numero")
        ancho_vetanatana = 250
        alto_vetanatana = 200
        ancho_pantalla = self.vetnanaSec.winfo_screenwidth()
        alto_pantalla = self.vetnanaSec.winfo_screenheight()
        x = (ancho_pantalla // 2) - (ancho_vetanatana // 2)
        y = (alto_pantalla // 2) - (alto_vetanatana // 2)
        self.vetnanaSec.geometry(f"{ancho_vetanatana}x{alto_vetanatana}+{x}+{y}")

        self.inicio()

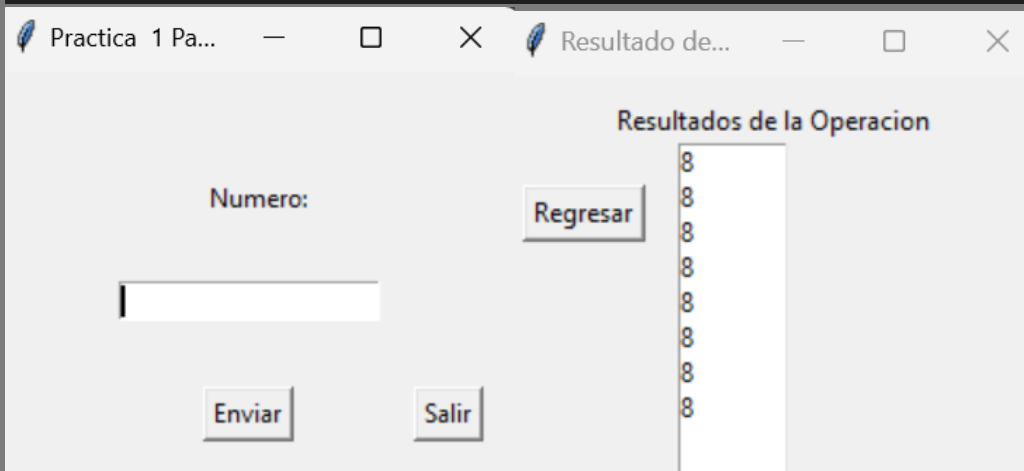
    def inicio(self):
        Label(self.vetnanaSec, text = "Resultados de la Operacion").place(x = 50, y = 10)
        self.miLista = Listbox(self.vetnanaSec, height = 10, width = 8, bg = "white")
        self.miLista.place(x = 80, y = 30)
        Button(self.vetnanaSec, text = "Regresar", command = self.volver).place(x = 10, y = 50)
        self.ingresarDatos()

    def ingresarDatos(self):
        for i in range(self.numero):
            self.miLista.insert(END, self.numero)

    def volver(self):
        self.vetnanaSec.destroy()
        self.ventaPrimaria.deiconify()

if __name__ == "__main__":
    master = Tk()
    app = Principal(master)
    app.inicio()
    master.mainloop()

```



PRACTICA 4

Tabla donde agregas producto, descripción, precio y cantidad agregando un ID y código el cual también puedes modificar o eliminar

```
#LIBRERIAS/LIBRARIES
from tkinter import *
from tkinter import messagebox
import sqlite3
from tkinter import ttk
import tkinter as tk
import random

#CREACION BASE DE DATOS/DATABASE CREATION
def crearBaseDatos():
    #CREA BASE DE DATOS tienda.db
    #CREATES DATABASE tienda.db
    con = sqlite3.connect("tienda.db")
    #CURSOR PARA EJECUTAR COMANDOS SQL
    #CURSOR TO EXECUTE SQL COMMANDS
    cursor = con.cursor()
    #CREA LA TABLA productos/CREATES TABLE productos
    cursor.execute("""CREATE TABLE IF NOT EXISTS productos(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        codigo TEXT NOT NULL,
        producto TEXT NOT NULL,
        precio REAL NOT NULL)""")
    #CREA LA TABLA almacen/CREATES TABLE almacen
    cursor.execute("""CREATE TABLE IF NOT EXISTS almacen(
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        codigoproducto TEXT NOT NULL,
        stock INTEGER NOT NULL,
        descripcion TEXT NOT NULL)""")
    con.commit()
    con.close()

#CLASE PRINCIPAL/MAIN CLASS
class Principal():
    #TAMAÑO DE VENTANA/WINDOW SIZE
    def __init__(self, master):
        self.ven = master
        self.ven.title('Practica 2 Parcial 3')
        self.ven.geometry("550x400")
        ancho = 550
        alto = 400
        ventana_alto = self.ven.winfo_screenwidth()
        ventana_ancho = self.ven.winfo_screenheight()
        x = (ventana_alto // 2) - (ancho // 2)
        y = (ventana_ancho // 2) - (alto // 2)
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")
        #ELIMINAR UN REGISTRO/DELETE A RECORD
        self.index = -1
```

```
#INTERFAZ DE LA VENTANA/WINDOW INTERFACE
def inicio(self):
    self.us=Label(self.ven, text=f"CRUD DE PRODUCTOS")
    self.us.place(x=50,y=5)
    Label(self.ven, text="Producto").place(y=30,x=10)
    self.producto = Entry(self.ven)
    self.producto.place(y=50,x=10)
    Label(self.ven, text="Descripcion").place(y=30,x=150)
    self.descripcion = Entry(self.ven)
    self.descripcion.place(y=50,x=150)
    Label(self.ven, text="Precio").place(y=30,x=290)
    self.precio = Entry(self.ven)
    self.precio.place(y=50,x=290)
    Label(self.ven, text="Cantidad").place(y=30,x=420)
    self.cantidad = Entry(self.ven)
    self.cantidad.place(y=50,x=420)
    #TITULOS DE LA TABLA CREADA/TABLE HEADERS CREATED
    columnas = ("ID","CODIGO","PRODUCTO","PRECIO","DESCRIPCION","STOCK")
    #CREAR EL TREEVIEW/CREATE THE TREEVIEW
    self.tabla = ttk.Treeview(self.ven, columns= columnas, show="headings")
    self.tabla.place(x=10, y=100, width=480,height=190)

#MUESTRA TEXTO DEL ENCABEZADO IMPORTANTE
#DISPLAYS HEADER TEXT
for col in columnas:
    self.tabla.heading(col,text=col)
    self.tabla.column(col, anchor="center", width=30)
#CREA BARRAS PARA DESPLAZARSE/CREATES SCROLLBARS
scrolly = ttk.Scrollbar(self.ven,orient="vertical", command=self.tabla.yview)
scrollx = ttk.Scrollbar(self.ven, orient="horizontal", command=self.tabla.xview)
scrolly.place(x=480,y=90,height=200)
scrollx.place(x=10,y=280, width=470)

self.agregar = Button(self.ven, text="Agregar", width=10, state="normal", command=self.agregarproducto)
self.agregar.place(x=30,y=320)

self.modificar = Button(self.ven, text="Modificar", width=10, state="disabled", command=self.modificarproducto)
self.modificar.place(x=120,y=320)

self.eliminar = Button(self.ven, text="Eliminar", width=10, state="disabled", command=self.eliminarproducto)
self.eliminar.place(x=210,y=320)

#SELECCIONAR UNA FILA/SELECT A ROW
self.tabla.bind("<<TreeviewSelect>>", self.seleccionfila)
#TRAER LOS DATOS DE LA BASE DE DATOS PARA AGREGARLOS AL TREEVIEW
#FETCH DATABASE DATA TO DISPLAY IN TREEVIEW
self.mostrarDatos()
```

```
def modificarproducto(self):
    #OBTENER LA FILA SELECCIONADA EN LA TABLA
    #GET THE SELECTED ROW FROM THE TABLE
    try:
        self.index = self.tabla.selection()[0]
    except:
        return
    #RECUPERA LOS VALORES DE LA FILA SELECCIONADA
    #RETRIEVE VALUES FROM SELECTED ROW
    valores = self.tabla.item(self.index,"values")
    #IDENTIFICADOR DEL PRODUCTO QUESE VA A MODIFICAR
    #IDENTIFIER OF PRODUCT TO MODIFY
    id = valores[0]
    #OBTIENE LOS DATOS INGRESADOS POR EL USUARIO EN LAS CAJAS DE TEXTO
    #GET DATA ENTERED BY USER
    pro = self.producto.get()
    pre = self.precio.get()
    des = self.descripcion.get()
    can = self.cantidad.get()
    #VERIFICA QUE NO TENGA CAJAS VACIAS
    #VERIFY THAT NO FIELDS ARE EMPTY

    if len(pro) != 0 and len(pre) != 0 and len(des) != 0 and len(can) != 0:
        codigo = pro[:2].upper() + str(random.randint(0,100)) + des[0].upper()
        #GENERA CODIGO: PRIMERAS 2 LETRAS DEL NOMBRE
        #          NUMERO ALEATORIO ENTRE 0 Y 100
        #          PRIMERA LETRA DE LA DESCRIPCION
        #GENERATE CODE: FIRST 2 LETTERS OF PRODUCT
        #          RANDOM NUMBER 0-100
        #          FIRST LETTER OF DESCRIPTION

        #CONECTA CON LA BASE DE DATOS/CONNECT TO DATABASE
        con = sqlite3.connect("tienda.db")
        cursor = con.cursor()
        #ACTUALIZA LOS DATOS DEL PRODUCTO EN PRODUCTO Y ALMACEN
        #UPDATE PRODUCT DATA IN productos AND almacen
        cursor.execute("UPDATE productos SET codigo=?, producto=?, precio=? WHERE id=?", (codigo,pro,pre,id))
        cursor.execute("UPDATE almacen SET codigoproducto=?, stock=?, descripcion=? WHERE id=?", (codigo,can,des,id))

        con.commit()
        con.close()
        self.limpiarcajas()
        self.actulizartable()
        #ACTIVA Y DESACTIVA LOS BOTONES
        #ENABLE/DISABLE BUTTONS
        self.agregar.config(state="normal")
        self.modificar.config(state="disabled")
        self.eliminar.config(state="disabled")
```

```
self.eliminar.config(state="disabled")
else:
    messagebox.showinfo("Error", "Faltan datos")

def eliminarproducto(self):
    try:
        self.index = self.tabla.selection()[0]
    except:
        return

    valores = self.tabla.item(self.index, "values")
    id = valores[0]

    con = sqlite3.connect("tienda.db")
    cursor = con.cursor()

    cursor.execute("DELETE FROM productos WHERE id=?", (id))
    cursor.execute("DELETE FROM almacen WHERE id=?", (id))

    con.commit()
    con.close()
    self.actulizartable()
    self.limpiarcajas()

    self.agregar.config(state="normal")
    self.modificar.config(state="disabled")
    self.eliminar.config(state="disabled")

def seleccionfila(self, event):
    self.limpiarcajas()
    try:
        self.index = self.tabla.selection()[0]
    except:
        return

    valores = self.tabla.item(self.index, "values")

    self.producto.insert(0, valores[2])
    self.precio.insert(0, valores[3])
    self.descripcion.insert(0, valores[4])
    self.cantidad.insert(0, valores[5])

    self.agregar.config(state="disabled")
    self.modificar.config(state="normal")
    self.eliminar.config(state="normal")
```

```
def mostrarDatos(self):
    con = sqlite3.connect("tienda.db")
    cursor = con.cursor()
    cursor.execute("""
        SELECT productos.id,
           productos.codigo,
           productos.producto,
           productos.precio,
           almacen.descripcion,
           almacen.stock
        FROM productos
        INNER JOIN almacen
        ON productos.codigo = almacen.codigoproducto
    """)

    for i in cursor.fetchall():
        self.tabla.insert("",END,values=i)
    con.commit()
    con.close()

def verificar(self):
    p = self.producto.get()
    d = self.descripcion.get()
    pr = self.precio.get()
    s = self.cantidad.get()

    if len(p)!=0 and len(d)!=0 and len(pr)!= 0 and len(s)!= 0:
        con = sqlite3.connect("tienda.db")
        cursor = con.cursor()
        #ABRE BASE DE DATOS Y EJECUTA UNA CONSULTA PARA BUSCAR
        #EL PRODUCTO QUE TENGA EL MISMO NOMBRE Y DESCRIPCION
        #OPENS DB AND EXECUTES A QUERY TO SEARCH
        #FOR A PRODUCT WITH SAME NAME AND DESCRIPTION
        cursor.execute("""
            SELECT productos.*, almacen.*
            FROM productos
            JOIN almacen ON productos.codigo = almacen.codigoproducto
            WHERE productos.producto = ? AND almacen.descripcion = ?
        """, (p, d))

        #DEVUELVE LA PRIMERA COINCIDENCIA ENCONTRANDO NONE SI NO EXISTE
        #RETURNS FIRST MATCH OR NONE IF NOT FOUND
        resultado = cursor.fetchone()
        con.commit()
        cursor.close()
        print(resultado)
        if not resultado:
            return False
        else:
            return resultado[0]
```

```
def agregarproducto(self):
    #REVISA SI YA EXISTE UN PRODUCTO CON EL MISMO NOMBRE
    #CHECKS IF PRODUCT WITH SAME NAME ALREADY EXISTS
    if self.verificar():
        id = self.verificar()
        print(id)

        p = self.producto.get()
        d = self.descripcion.get()
        pr = self.precio.get()
        s = self.cantidad.get()

        if len(p)!=0 and len(d)!=0 and len(pr)!= 0 and len(s)!= 0:
            con = sqlite3.connect("tienda.db")
            cursor = con.cursor()
            #ACTUALIZA PRECIO (PRODUCTOS) Y STOCK (ALMACEN)
            #UPDATES PRICE (productos) AND STOCK (almacen)
            cursor.execute("UPDATE productos SET precio=? WHERE producto=?", (pr,p))
            cursor.execute("UPDATE almacen SET stock=? WHERE descripcion=?", (s,d))

            con.commit()
            con.close()
            self.limpiarcajas()
            self.actulizartable()

    else:
        #CONVIERTE PRECIO FLOAT Y CANTIDAD ENTERO
        #CONVERTS PRICE TO FLOAT AND QUANTITY TO INT
        prf = 0.0
        st = 0

        p = self.producto.get()
        d = self.descripcion.get()
        pr = self.precio.get()
        s = self.cantidad.get()

    if len(p)!=0 and len(d)!=0 and len(pr)!= 0 and len(s)!= 0:
        prf = float(pr)
        st = int(s)
        #GENERA CODIGO ALEATORIO PARA EL PRODUCTO
        #GENERATES RANDOM PRODUCT CODE
        codigo = p[:2].upper() + str(random.randint(0,100)) + d[0].upper()
        con = sqlite3.connect("tienda.db")
        cursor = con.cursor()
        #INSERTA LOS DATOS EN LAS TABLAS
        #INSERTS THE DATA INTO TABLES
        cursor.execute("INSERT INTO productos (codigo,producto,precio) VALUES (?, ?, ?)", (codigo, p, prf))
        cursor.execute("INSERT INTO almacen (codigoproducto,descripcion,stock) VALUES (?, ?, ?)", (codigo,d,st))
        con.commit()
        con.close()
        self.actulizartable()
        self.limpiarcajas()
    else:
        messagebox.showerror("ERROR", "Faltan datos")
```

```

def limpiarcajas(self):
    self.producto.delete(0,END)
    self.precio.delete(0,END)
    self.descripcion.delete(0,END)
    self.cantidad.delete(0,END)

def actualizartable(self):
    for i in self.tabla.get_children():
        self.tabla.delete(i)
    self.mostrarDatos()

if __name__=='__main__':
    crearBaseDatos()
    master = Tk()
    app = Principal(master)
    app.inicio()
    master.mainloop()

```

Practica 2 Parcial 3

CRUD DE PRODUCTOS

Producto Descripción Precio Cantidad

ID	CODIGO	PRODUCTO	PRECIO	DESCRIPCION	STOCK
2	JA9P	jabon	98.0	polvo	2
3	LE46D	leche	60.0	deslactosada	2

Agregar Modificar Eliminar

Filter 3 tables... Rows: 2 Filter 2 rows... Upgrade to PRO

TABLES		id	codigopr...	stock	descripci...
> almacen		id	codigopr...	stock	descripci...
> productos		id	codigopr...	stock	descripci...
> sqlite_sequence		id	codigopr...	stock	descripci...
	1	2	JA9P	2	polvo
	2	3	LE46D	2	deslactosada
	+	3			

EXTRAS

Operaciones en python

ARIRMETICAS HACEN CALCULOS			
+	SUMA	$3 + 2 = 5$	
-	RESTA	$5 - 2 = 3$	
*	MULTIPLICACION	$4 * 3 = 12$	
/	DIVISION DECIMAL	$7 / 2 = 3.5$	Siempre devuelve un número decimal (float).
//	DIVISION SIN DECIMAL	$7 // 2 = 3$	Devuelve solo la parte entera (redondea hacia abajo)
%	MODULO	$7 \% 2 = 1$ $N \% 2 = 0$ par $N \% 2 == 1$ impar	Devuelve el resto de una división. Se usa para Números Primos
**	POTENCIA	$5 ** 2 = 25$ (5 al cuadrado)	
$** (1/2)$	RAIZ CUADRADA	$9 ** (1/2) = 3.0$ (Raiz cuadrada de 9)	
$** (1/3)$	RAIZ CUBICA	$27 ** (1/3) = 3.0$ (Raiz cubica de 27)	
OPERADORES LOGICOS COMBINAN CONDICIONES BOOLEANAS (TRUE / FALSE)			
and	LOGICAL AND	$(5 > 2)$ and $(3 < 4) = \text{True}$	Devuelve True solo si ambas condiciones son verdaderas
or	LOGICAL OR	$(5 > 10)$ or $(3 < 4) = \text{True}$	Devuelve True si al menos una condición es verdadera.
not	NEGACION LOGICA	Not True = False Not $(5 > 2) = \text{False}$	Invierte el valor de verdad
OPERADORES RELACIONALES COMPARAN VALORES			
<	MENOR QUE	$3 < 5 = \text{True}$	
>	MAYOR QUE	$7 > 10 = \text{False}$	
<=	MENOR O IGUAL QUE	$4 <= 4 = \text{True}$	
>=	MAYOR O IGUAL QUE	$6 >= 5 = \text{True}$	
==	IGUAL A	$5 == 5 = \text{True}$	Comprueba si dos valores son iguales
!=	DIFERENTE DE	$5 != 3 = \text{True}$	

i	ELEMENTO INDIVIDUAL	Cuando recorres una lista con un bucle for, cada valor se guarda en la variable i	<pre> 1 frutas = ["manzana", "pera", "uva"] 2 3 for i in frutas: 4 # i es cada fruta individual 5 print(f"Elemento individual: {i}") </pre> Elemento individual: manzana Elemento individual: pera Elemento individual: uva
Ord	Valor ascii	La función ord() devuelve el código ASCII/Unicode de un carácter:	
append	Agrega elemento en la lista		

INSTRUCCIONES ENTRADA Y SALIDA

<code>print()</code>	<code>Print('Hola Mundo')</code>	Mostrar mensajes en pantalla
<code>print(f'....')</code>	numero = 10 <code>print(f'Hola Mundo, numero: {numero}')</code> Salida: Hola Mundo, numero 10	La f antes de las comillas permite insertar variables o valores dentro de la cadena usando {}
<code>input('....')</code>	nombre = <code>input('Escribe tu nombre: ')</code> <code>print(f'Hola {nombre}')</code> Salida: Hola Osiris	

CASTING PARA CONVERTIR A VALORES ESPECIFICOS FLOAT INT

f = 0.0 f = float(input('Escribe numero DECIMAL')) Y de decimal a entero	<code>f = float(input('Escribe un numero decimal: '))</code> <code>print(f'Tu numero decimal es {f}')</code> Salida: Escribe un numero decimal: 9.9 Tu numero decimal es 9.9 <code>f = float(input('Escribe un numero decimal: '))</code> <code>entero = int(f)</code> <code>print(f'Tu numero decimal es {f} y entero {entero}')</code> Salida: Escribe un numero decimal: 9.9 Tu numero decimal es 9.9 y entero 9	
a = 0 a = int(input('Escribe numero ENTERO')) Y entero a decimal	<code>a = int(input('Escribe un numero entero: '))</code> <code>print(f'Tu numero entero es {a}')</code> Salida: Escribe un numero entero 9 Tu numero entero es 9 <code>a = int(input('Escribe un numero entero: '))</code> <code>decimal = float(a)</code> <code>print(f'Tu numero entero es {a} y decimal {decimal}')</code> Salida: Escribe un numero entero: 9 Tu numero entero es 9 y decimal 9.0	
Cadena = 120 Print(str(cadena))	Cadena = 120 numero = 120 Print(str(cadena)) v = str(numero) Print(v) Salida: 120 Salida: 120	

En Python **solo es obligatorio inicializar las variables que no provienen del teclado** si quieres usarlas antes de asignarles un valor.

Variable obligatoria si no viene de input
 x = 0

Variable que recibe input NO necesita inicializarla
 y = int(input('Escribe un número: '))
 break"

PARA QUE MARQUE SOLO 2 DECIMALES

{Cantidad:.2f}

CICLO INTRODUCIR LOS DATOS QUE TU QUIERAS CON OPCION DE SI O NO PARA SEGUIR

While(True):

respuesta = input('Deseas otro numero (s/n): \n')

if respuesta == 'n' or respuesta == 'N':

break

INTRODUCIR LOS DATOS QUE TE PIDA EL PROGRAMA COMO RANGOS TIENES QUE QUITAR ESTAS 3 LINEAS DE CODIGO

for i range(0,5):

respuesta = input('Deseas otro numero (s/n): \n')

if respuesta == 'n' or respuesta == 'N':

break

Isalpha()	Es un método de strings que verifica si todos los caracteres son letras (A-Z, a-z). Devuelve True si son todas letras, False si hay números, espacios o símbolos	<pre>texto = "Hola" print(texto.isalpha())</pre> Salida: True <pre>texto2 = "Hola123" print(texto2.isalpha())</pre> Salida: False
Isdigit()	Es un método de strings que verifica si todos los caracteres son números. Devuelve True si todos son dígitos, False si hay letras o símbolos. Tampoco valida en punto	<pre>num = "12345" print(num.isdigit())</pre> Salida: True <pre>num2 = "123a" print(num2.isdigit())</pre> Salida: False
Try: Except ValueError:	Sirve para manejar errores cuando intentas convertir un valor o hacer una operación que podría fallar. ValueError ocurre, por ejemplo, al convertir una letra a entero o float.	<pre>valor = input("Escribe un número: ") try: numero = int(valor) print(f"Tu número es {numero}") except ValueError: print("Eso no es un número válido")</pre> Salida: Escribe un numero: 10 Tu numero es 10 Salida: Escribe un numero: Hola Eso no es un numero valido

REPASO

OBTENER PRIMERA LETRA EN MAYUSCULA	texto.title() Pone la primera letra de cada palabra en mayúscula	<pre>nombre = input("Ingresa un nombre: ") nom = nombre.capitalize() print(f"Nombre es: {nom}")</pre>
------------------------------------	--	---

	texto.upper() Todo en mayúsculas texto.lower() Todo en minúsculas	
OBTENER LA CURP		<pre> nombre = input("Nombre: ").strip().upper() paterno = input("Apellido Paterno: ").strip().upper() materno = input("Apellido Materno: ").strip().upper() anio = int(input("Año de nacimiento: ")) mes = int(input("Mes: ")) dia = int(input("Día: ")) sexo = input("Sexo (H/M): ").upper() estado = input("Estado: ").strip().upper() if len(estado) >= 2: esta2 = estado[0] + estado[-2] else: esta2 = estado curp = (paterno[0] + paterno[1] + materno[0] + nombre [0] + anio [2:] + mes + dia + sexo + esta2) print(f"La CURP es: {curp}") </pre>
OBTENER RFC		<pre> nombre = input("Nombre: ").strip().upper() paterno = input("Apellido paterno: ").strip().upper() materno = input("Apellido materno: ").strip().upper() anio = input("Año (YYYY): ") mes = input("Mes (MM): ") dia = input("Día (DD): ") rfc = (paterno[2:] + materno[0] + nombre[0] + anio[2:] + mes + dia) print(f"RFC generado: {rfc}") </pre>
OPERACIONES		<pre> a = float(input("Número 1: ")) b = float(input("Número 2: ")) print(f"Suma: {a + b}") print(f"Resta: {a - b}") print(f"Multiplicación: {a * b}") print(f"División: {a / b:.2f}") # dos decimales print(f"División entera: {a // b}") print(f"Módulo: {a % b}") print(f"Potencia: {a ** 2}") # al cuadrado </pre>
SI TU EDAD ES NUMERO PRIMO		<pre> nombre = input("Ingresa tu nombre: ").strip().capitalize() edad_str = input("Ingresa tu edad: ") if edad_str.isdigit(): edad = int(edad_str) # Validar si la edad es primo primo = True if edad <= 1: primo = False else: for i in range(2, edad): if edad % i == 0: primo = False break print(f"Hola {nombre}, tu edad es {edad}.") if primo: print("Tu edad es un número primo") else: print("Tu edad NO es un número primo.") else: print("Edad no válida, debes ingresar un número.") </pre>
FORMA ASCENDENTE Y DESCENDENTE	sorted(lista) devuelve una lista ordenada (no cambia la original). reverse=True ordena de	<pre> numeros = [] for i in range(5): n = int(input(f"Ingresa número {i+1}: ")) numeros.append(n) print("\nLista original:", numeros) print("Ascendente:", sorted(numeros)) print("Descendente:", sorted(numeros, reverse=True)) </pre>

	mayor a menor.	
INGRESAR NOMBRES, MUESTRA EN LISTA Y SUMA EL TOTAL DE NOMBRES INGRESADOS		<pre>nombres = [] while True: nombre = input("Ingresa un nombre (o 'fin' para salir): ").capitalize() if nombre.lower() == "fin": break nombres.append(nombre) print("\nLista de nombres:", nombres) print(f"Total de nombres: {len(nombres)}") lista = ["Ana", "Luis", "Pedro", "Marta"] print("Lista actual:", lista)</pre>
ELIMINAR UN NOMBRE DE LA LISTA		<pre>buscar = input("¿Qué nombre deseas eliminar?: ").capitalize() if buscar in lista: lista.remove(buscar) print(f"{buscar} eliminado correctamente.") else: print("Ese nombre no está en la lista.") print("Lista actualizada:", lista)</pre>
MUESTRA LISTA DE PERSONAS CON EDAD Y SEXO		<pre>personas = [] for i in range(3): nombre = input("Nombre: ").capitalize() edad = int(input("Edad: ")) sexo = input("Sexo (H/M): ").upper() personas.append({"nombre": nombre, "edad": edad, "sexo": sexo}) print("\nLista de personas:") for p in personas: print(f"{p['nombre']} - {p['edad']} años - {p['sexo']}")</pre>
LEER UNA CADENA Y CONTAR CUÁNTAS VOCALES TIENE		<pre>texto = input("Escribe una cadena: ").lower() vocales = "aeiou" contador = 0 for letra in texto: if letra in vocales: contador += 1 print(f"La cadena tiene {contador} vocales.")</pre>
LEER UNA LISTA DE NOMBRES Y MOSTRAR CUÁNTOS EMPIEZAN CON VOCAL		<pre>nombres = [input(f"Nombre {i+1}: ").strip().capitalize() for i in range(5)] vocales = "AEIOU" contador = sum(1 for n in nombres if n[0] in vocales) print(f"{contador} nombres comienzan con vocal.")</pre>
PEDIR 5 CALIFICACIONES Y MOSTRAR LISTA, PROMEDIO, MAYOR Y MENOR		<pre>bloweqto = max(csjtttcsctoue) - min(csjtttcsctoue) bloweqto = (bloweqto / 5) bloweqto = round(csjtttcsctoue, 2) bloweqto = sum(csjtttcsctoue) / len(csjtttcsctoue) csjtttcsctoue = sorted(csjtttcsctoue) bloweqto = (csjtttcsctoue[0] + csjtttcsctoue[-1]) / 2</pre>
PALINDROMO		<pre>palabra = input("Escribe una palabra: ").lower().replace(" ", "") if palabra == palabra[::-1]: print("Es un palíndromo.") else: print("No es un palíndromo.")</pre>
PALABRAS CON TODAS LA VOCALES		<pre>palabra = input("Escribe una palabra: ").lower() vocales = "aeiou" if all(v in palabra for v in vocales): print("La palabra contiene todas las vocales.") else: print("Faltan vocales.")</pre>

MOSTRAR LAS INICIALES DE UN NOMBRE		<pre> nombre = input("Escribe tu nombre completo: ").title().split() iniciales = "".join([n[0] for n in nombre]) print("Iniciales:", iniciales) </pre>
CALCULAR EL PORCENTAJE DE HOMBRES Y MUJERES EN UNA LISTA		<pre> personas = ["H", "M", "H", "M", "H"] total = len(personas) h = personas.count("H") m = personas.count("M") print(f"Hombres: {h/total*100:.1f}%") print(f"Mujeres: {m/total*100:.1f}%") </pre>

upper() para convertir todo a mayúsculas

lower() para convertir todo a minúsculas

swapcase() para invertir el caso de las letras

capitalize() para poner en mayúscula solo la primera letra de una cadena

title() para poner en mayúscula la primera letra de cada palabra.

.split(",") Toma una cadena de texto y la divide en partes, creando una lista

.strip() Elimina espacios en blanco al principio y al final de un texto.

.get():

GUIA DE CODIGOS

❖ El **Listbox** sirve para mostrar elementos en una lista dentro de la ventana

```
self.lista = Listbox(self.ven, height=10, width=10, bg="white", font=("Helvetica",12))
```

```
self.lista.place(x=190, y=10)
```

❖ Acciones comunes

```
self.lista.insert(END, valor)
```

Agrega al final

```
self.lista.delete(0, END)
```

Borra todo

```
self.lista.size()
```

Devuelve cuántos elementos hay

```
self.lista.delete(self.lista.size()-1)
```

Borra el último

Usado en Pilas y Colas para representar visualmente la estructura.

❖ **Pilas y Colas** (estructura de datos)

PILA

Último que entra, primero que sale.

```
self.datos.append(rfc)
```

#Agregar (PILA)

```
self.datos.pop()
```

Eliminar (PILA)

Ejemplo en ListBox

```
self.lista.delete(self.lista.size()-1)
```

COLA

Primero que entra, primero que sale.

```
self.datos.insert(0, rfc)
```

```
self.datos.pop(0)
```

```
# Agregar (COLA)
```

```
# Eliminar (COLA)
```

Ejemplo de ListBox

```
self.lista.delete(0)
```

❖ Radiobuttons (botones de opción única)

```
self.modos = StringVar(value="Pilas")
```

```
Radiobutton(self.ven, text="Pilas", variable=self.modos, value="Pilas").place(x=50, y=40)
```

```
Radiobutton(self.ven, text="Colas", variable=self.modos, value="Colas").place(x=100, y=40)
```

❖ Validaciones

```
if not self.val.ValidarLetra(nombre):
```

```
    messagebox.showerror('Error', 'El nombre solo debe contener letras')
```

```
if not self.val.ValidarNumeros(telefono):
```

```
    messagebox.showerror('Error', 'El teléfono solo debe contener números')
```

Segunda hoja validaciones

Correo@ y punto

```
def ValidarCorreo(self, correo):
```

```
    if "@" not in correo or "." not in correo:
```

```
        return False
```

```
    partes = correo.split("@")
```

```
    if len(partes) != 2:
```

```
        return False
```

```
    local, dominio = partes
```

```
    if not local or not dominio:
```

```
        return False
```

```
def ValidarDominio(d):
```

```
    if "." not in d:
```

```
        return len(d) >= 2
```

```
else:
```

```
    parte, resto = d.split(".", 1)
```

```
    if not parte:
```

```
        return False
```

```
    return ValidarDominio(resto)
```

```
return ValidarDominio(dominio)
```

❖ Ordenamientos (Burbuja y Selección)

BURBUJA

Compara pares y los intercambia si están en desorden. Ordena de menor a mayor.

```
for i in range(0, len(self.lis)):
```

```
    for x in range(i, len(self.lis)-1):
```

```
        if self.lis[x] > self.lis[x+1]:
```

```
            aux = self.lis[x]
```

```
            self.lis[x] = self.lis[x+1]
```

```
            self.lis[x+1] = aux
```

SELECCIÓN

Busca el número mayor o menor y lo coloca en su lugar. Ordena de mayor a menor.

```
for i in range(0, len(self.lis)):
```

```
    aux = int(self.lis[i])
```

```
    p = i
```

```
    for x in range(i, len(self.lis)):
```

```
        if aux < int(self.lis[x]): # Si el siguiente es mayor
```

```
            aux = int(self.lis[x])
```

```
            p = x
```

```
self.lis[p] = self.lis[i]
```

```
self.lis[i] = str(aux)
```

❖ Placeholders en Entry (Texto guía)

Sirve para mostrar texto dentro del Entry como guía ("Nombre", "Teléfono", etc.).

```
self.placeholder1 = "Nombre"
```

```
self.nombre = Entry(self.ven, fg="gray")
```

```
self.nombre.insert(0, self.placeholder1)
```

```
self.nombre.bind("<FocusIn>", self.quitar_placeholder1)
```

```
self.nombre.bind("<FocusOut>", self.poner_placeholder1)
```

```
Nombre Telefono Domicilio
```

❖ Treeview (Tabla tipo Excel)

Usado para mostrar varios campos (clave, nombre, correo, edad).

```
columnas = ("Clave", "Nombre", "Correo", "Edad")
```

```
self.tabla = ttk.Treeview(self.ven, columns=columnas, show="headings")
```

Agregar un dato

```
self.tabla.insert("", "end", values=(clave, nombre, correo, edad))
```

eliminar fila

```
self.tabla.delete(renglon)
```

seleccionar fila

```
self.renglon = self.tabla.selection()
```

```
valores = self.tabla.item(self.renglon, "values")
```

Variable	Significado
<code>self.modos</code>	Guarda si es Pila o Cola
<code>self.lis</code>	Lista temporal para ordenar
<code>self.val</code>	Llama a clase Validar
<code>self.lista</code>	Listbox visual
<code>self.tabla</code>	Tabla Treeview
<code>self.modosorden</code>	Guarda el método de ordenamiento

❖ Mensajes

```
messagebox.showerror("Error", "La lista está vacía")
```

```
messagebox.showinfo("Correcto", "RFC agregado con éxito")
```



```
messagebox.showwarning("Atención", "No hay elementos para eliminar.")
```

1. INICIO

```
from tkinter import *
from tkinter import messagebox
from Validacionesp1 import Validar

class Principal():
    def __init__(self):
        self.val = Validar()
        self.ven = Tk()
        ancho = 320
        alto = 210
        ventana_alto = self.ven.winfo_screenwidth()
        ventana_ancho = self.ven.winfo_screenheight()
        x = (ventana_alto // 2) - (ancho // 2)
        y = (ventana_ancho // 2) - (alto // 2)
        self.ven.geometry(f"{ancho}x{alto}+{x}+{y-100}")

    def inicio(self):
        self.ven.mainloop()

if __name__ == '__main__':
    app = Principal()
    app.inicio()
```

2. TEXTO

```
self.label = Label(text="")
self.label.place(x=5, y=80)
```

3. CAJA DE TEXTO

```
self.dato = Entry(self.ven)
self.dato.place(x=50, y=10)
```

VALIDACIÓN (intentos de convertir a número)

<pre>try: caja1 = int(self.n1.get()) caja2 = int(self.n2.get()) except ValueError: messagebox.showerror("Error", "Algún dato no es número")</pre>	Crear ventana Poner título Definir tamaño y posición
---	--

VALIDACIÓN DE DATOS

```
if len(self.num1.get()) <= 0:
    messagebox.showerror("Error", "Campo vacío")
    return
```

CREACIÓN DE VENTANAS

```
Master = Tk()
self.vetana = master
self.vetana.title("Practica 1 Parcial 3")
self.vetana.geometry("250x200+centro")
```

VENTANA SECUNDARIA (TOPLEVEL)

```
self.vetana.withdraw()
otra = Toplevel(self.vetana)
Ventana2(otra, self.vetana, caja1, caja2)
```

LISTBOX PARA MOSTRAR DATOS

```
self.miLista = Listbox(self.vetnanaSec, height=10, width=8)
self.miLista.place(x=80, y=30)
Crear Lista, Mostrar valores
```

INGRESO DE DATOS AL LISTBOX

```
for i in range(self.numero):
    self.miLista.insert(END, self.numero)
Llenar una lista con un número repetido
Uso de insert()
```

OCULTAR Y MOSTRAR VENTANAS

```
self.vetana.withdraw() # Oculta ventana principal
self.vetana.deiconify() # Muestra ventana principal
self.venDos.destroy() # Cierra ventana secundaria
```

MÉTODOS IMPORTANTE: REGRESAR Y CERRAR

```
def regresar(self):
    self.venDos.destroy() Cerrar ventana secundaria
    self.vetana.deiconify() Regresar a la principal
```

CERRAR PROGRAMA COMPLETO

```
def cerrar(self):
    self.vetana.destroy()
```

CREACIÓN DE LA BASE DE DATOS

```
CREATE TABLE productos(id, codigo, producto, precio)
CREATE TABLE almacen(id, codigoproducto, stock, descripcion)
```

TABLA (Treeview)

```
self.tabla = ttk.Treeview(columns=(...), show="headings")
self.tabla.insert("", END, values=i)
```

Muestra ID, código, nombre, precio, descripción y stock
 Permite seleccionar filas
 Al seleccionar una fila → llena las cajas de texto

GENERAR CODIGO

```
codigo = p[:2].upper() + str(random.randint(0,100)) + d[0].upper()
```

MOSTRAR DATOS EN LA TABLA

```
SELECT productos.id, productos.codigo, productos.producto,
       productos.precio, almacen.descripcion, almacen.stock
FROM productos
INNER JOIN almacen
ON productos.codigo = almacen.codigoproducto
```

LIMPIAR Y ACTUALIZAR TABLA

```
def actualizartable(self):
    for i in self.tabla.get_children():
        self.tabla.delete(i)
    self.mostrarDatos()
```

SUMA DE NÚMEROS

```
def sumar(self):
    messagebox.showinfo("Suma", f"La suma de {self.c1} y {self.c2} es: {self.c1 + self.c2}")
```

INICIO DE VENTANA

```
class Principal():
    def __init__(self, master):
        pass

    def inicio(self):
        pass

if __name__ == "__main__":
    master = Tk()
    app = Principal(master)
    app.inicio()
    master.mainloop()
```

TAMAÑO Y TÍTULO

```
self.ventana = master
self.ventana.title("REPASO")
ancho_ventana = 250
alto_ventana = 200
```

```
ancho_pantalla = self.ventana.winfo_screenwidth()
alto_pantalla = self.ventana.winfo_screenheight()
x = (ancho_pantalla // 2) - (ancho_ventana // 2)
y = (alto_pantalla // 2) - (alto_ventana // 2)
self.ventana.geometry(f"{ancho_ventana}x{alto_ventana}+{x}+{y}")
```

NUMEROS RANDOM

```
f"{random.randint(0,999):03}"
```